IACE PLATFORM · OPERATIONS · SECOND EDITION

Staging deployment runbook

Standing the API, the database, the cache and the two websites up on AWS — and running them afterwards.

Region ap-south-1 (Mumbai) · **Shape** EC2 + Docker Compose behind Caddy
Written 4 October 2026, after the first environment was built
Source of record docs/04-infrastructure.md, deploy/

Contents

1. What you are building, in plain words 4
2. The concepts, each in a paragraph 5
3. Before you start 7
4. AWS groundwork 9
5. RDS — the database 12
6. Box B — Valkey 16
7. Box A — the API box 21
8. DNS at GoDaddy 25
9. Bring the stack up 27
10. Seed the first super admin 29
11. The websites — S3 and CloudFront 30
12. Verify, end to end 38
13. Deploying after today — the pipeline 40
14. Caddy — what it is, and every line of the Caddyfile 42
15. Grafana Cloud — logs and metrics 44
16. Scaling 46
17. PostgreSQL — backups and restore 47
18. Valkey — connecting and looking inside 48
19. Production — what is different 49
20. Load testing 52
21. The SMS vendor 54
22. Email 55
23. When something is wrong 56
24. After today 57

Staging deployment runbook

IACE platform — standing it up on AWS, and running it afterwards

Written for somebody who has not deployed to AWS before. Every step says what to click or type, and then why — because the "why" is what lets you fix it when the step does not work, and something always does not work.

This is the second edition, written after the first environment was actually built. The real resource identifiers are in it, and so is every trap that cost time: the ones you would not guess, and the three places the first edition was simply wrong.

Read §1 and §2 first — twenty minutes, and the difference between following instructions and knowing what you are doing. Then §4 to §12 in order, which is the deployment. §13 onwards is reference: the pipeline, Caddy, Grafana, load testing, scaling, backups, production, SMS and email.

Where the authority lives. This is a runbook, not the source of record. The sizing and the money are `docs/04-infrastructure.md`. The data model is `prisma/schema.prisma`. The operational short form is `deploy/README.md`. Where this and one of those disagree, they win and this is stale.

---

1. What you are building, in plain words

Three computers and a managed database. That is the whole thing.

	What it is	What runs on it
Box A	One EC2 virtual machine	Caddy, the API in three containers, a telemetry shipper
Box B	A second, smaller EC2	Valkey — the in-memory store holding live exam sittings
RDS	PostgreSQL that AWS runs for you	Every durable row: students, tests, questions, marks
S3 + CloudFront	Object storage with a CDN in front	The two websites, and question images

A student's browser does two unrelated things at once. It downloads the **website** — HTML, JavaScript, CSS — from CloudFront, a cache spread across many cities. And it makes **API calls** — sign in, start test, save answers, submit — to Box A. Two addresses, two systems, and almost every deployment confusion comes from conflating them.

HOW A REQUEST REACHES THE RIGHT PLACE

Student's browser

DNS lookup at GoDaddy

Two destinations

website files → CloudFront → S3

API calls → Box A → Caddy

Inside Box A

Caddy routes by path

exam container → the live sitting

core container → everything else

worker container → scoring, no HTTP

alloy → logs and metrics out

Data

Valkey on Box B → answers in flight

RDS PostgreSQL → marks, for ever

Why three containers from one image. Node.js runs your code on one CPU core. A single container would use one core of a two-core box and leave the other idle for ever. So the same image starts three times with a different `API_ROLE`, and each answers a different set of routes. `exam` takes the live sitting — starting a paper, saving answers, submitting. `core` takes everything else: sign-in, results, the whole admin app. `worker` answers no web requests at all; it sits on a queue and scores papers after they are submitted.

Why the worker is exactly one, always. It runs a flush every 60 seconds and a sweep every 120, and `concurrency: 1` in `QUEUE_POLICY` is the only thing stopping one tick overlapping the next — which works inside a single process and nowhere else. Two worker containers put two sweeps over the same keys with nothing coordinating them. `exam` and `core` scale freely. The worker does not.

2. The concepts, each in a paragraph

You do not need these deeply. You need them well enough that an error message means something.

Region and availability zone. AWS runs data centres in groups. `ap-south-1` is Mumbai; inside it `ap-south-1a` and `ap-south-1b` are separate buildings. Everything here lives in one zone, because cross-zone traffic costs money and buys availability you are not yet using. You will still create a subnet in a second zone, for one reason: RDS refuses to be created without two.

VPC and subnets. A VPC is your own network inside AWS with its own address range — `10.0.0.0/16`, every address from `10.0.0.0` to `10.0.255.255`. A subnet is a slice of it pinned to one zone. A **public** subnet is one whose route table sends unknown traffic to an Internet Gateway. That is the only difference, and it does not by itself make anything reachable: three things must line up before a packet from the internet arrives — a public address on the interface, a security group rule, and a network ACL.

Security groups. A stateful firewall attached to a machine, not a subnet. Stateful means you write only the inbound rule and the reply is allowed automatically. The important trick: a rule's source can be **another security group** rather than an address range. Box B's rule says "allow 6379 from whatever is in the API group", so a replacement API box is trusted by joining the group and you never edit Box B.

Elastic IP. A public IPv4 address that belongs to you rather than to an instance, so it survives the instance being destroyed. It is also the failover mechanism: launch a replacement, move the address, and DNS never changes.

Instance profile. An IAM role attached to a machine. Software on it asks a local address for temporary credentials and gets them, scoped to that role. This is why no AWS key pair appears in your environment file — the box proves who it is by being itself.

Docker image versus container. An image is a frozen filesystem: your built app, Node, and its libraries. A container is one running copy. One image started three times with different settings gives the three roles. `docker compose` is a file describing several containers so one command starts them all.

Compose "service" versus "container". In `deploy/compose.yml`, `exam` is a *service*. Scale it to four and you get four *containers* all answering to the DNS name `exam` on Docker's private network. That is what makes scaling invisible to Caddy.

Reverse proxy. A program that accepts a request from the internet and forwards it to one of several programs behind it. Caddy is yours. It also terminates TLS: it obtains the certificate, decrypts, and speaks plain HTTP to your containers on the private network.

ACME and Let's Encrypt. Let's Encrypt issues certificates free and automatically. The HTTP-01 challenge: Caddy asks for a certificate for `api.staging.examprep.iace.co.in`; Let's Encrypt says "prove it — serve this token at `http://…/.well-known/acme-challenge/<id>`"; Caddy serves it; Let's Encrypt fetches it from the public internet and issues. **So the name must resolve and port 80 must be open before Caddy starts.** It is also why an AWS-provided hostname cannot be used: `*.amazonaws.com` is on the Public Suffix List and Let's Encrypt refuses it outright.

DNS record types. An **A** record maps a name to an IPv4 address. A **CNAME** maps a name to another name. You cannot put a CNAME at the root of a domain — but every name here is a subdomain, so that limit never bites.

TTL. How long the rest of the internet may remember a DNS answer. Keep it low (600) while setting up, so a mistake costs ten minutes rather than a day.

S3 and CloudFront. S3 stores files. CloudFront caches them at edges in many cities and brings a free certificate on its own `*.cloudfront.net` name. The websites are only files, so this is all they need — nothing renders them on a server.

RDS. AWS runs PostgreSQL for you: backups, point-in-time restore, storage autoscaling. This platform could run Postgres in a container and should not — the one operation nobody should perform for the first time under pressure is the one holding students' marks.

Valkey. A fork of Redis, same protocol. It holds a live sitting's answers while the exam runs, the session tokens, the OTP codes and the job queue. In memory, which is why answers go there every 25 seconds instead of to Postgres on every keystroke.

BullMQ. The job queue, stored in Valkey. On submit the API enqueues "score this attempt" and answers the student immediately; the worker picks it up a moment later. That is how 8,000 students can submit in the same minute without the database melting.

---

3. Before you start

3.1 Tools

```
curl "https://awscli.amazonaws.com/AWSCLIV2.pkg" -o "/tmp/AWSCLIV2.pkg"
sudo installer -pkg /tmp/AWSCLIV2.pkg -target /
aws --version
```

```
brew install --cask session-manager-plugin
session-manager-plugin
```

macOS has no "bundle" zip for the plugin, whatever a Linux guide says — the only official artifact is a signed `.pkg`, and the bundle URL returns an S3 `AccessDenied` document that `unzip` then reports as a corrupt archive. Without Homebrew:

```
curl -o /tmp/session-manager-plugin.pkg \
  "https://s3.amazonaws.com/session-manager-downloads/plugin/latest/mac_arm64/session-manager-plugin.pkg"
sudo installer -pkg /tmp/session-manager-plugin.pkg -target /
sudo ln -sf /usr/local/sessionmanagerplugin/bin/session-manager-plugin /usr/local/bin/session-manager-plugin
```

`mac_arm64` is Apple Silicon; plain `mac` is Intel. **A fallback worth knowing:** the EC2 console's **Connect → Session Manager** tab opens the same shell with nothing installed.

3.2 The CLI

IAM → Users → your user → Security credentials → Create access key → *Command Line Interface*.

```
aws configure
aws sts get-caller-identity
```

Billing may be invisible to you, and that is not a misconfiguration. An account-level switch, "IAM user and role access to billing information", is off by default and gates Cost Explorer and Budgets for *every* IAM user regardless of policy — `AdministratorAccess` does not open it. **Only the root user can turn it on**, so adding permissions to your user will not help. Ask for it: root sign-in → Account → IAM user and role access to billing information → Edit → Activate.

3.3 The names

The product is `examprep`, under the institute's existing `iace.co.in`. One label for the product, one for the environment, one for the service.

```
                api . staging . examprep . iace.co.in
                 |      |          |          |
             service   environment product   the zone you already own
```

Production drops the environment label, so the name typed most often is shortest.

Thing	Staging	Production
API	`api.staging.examprep.iace.co.in`	`api.examprep.iace.co.in`
Student site	`staging.examprep.iace.co.in`	`examprep.iace.co.in`

Thing	Staging	Production
Admin site	`admin.staging.examprep.iace.co.in`	`admin.examprep.iace.co.in`
SPA bucket	`examprep-staging-spas`	`examprep-prod-spas`
Media bucket	`examprep-staging-media`	`examprep-prod-media`
RDS instance	`examprep`	second database on it
Database	`examprep_staging`	`examprep_production`
Valkey port	6380	6379

> `examprep.iace.co.in` **is already taken.** It is a CNAME to `exam.thinkexam.in` — the platform this one replaces. Production's student hostname is therefore not a record to add but one to **repoint, with students already on it**: a scheduled cutover with the incumbent reachable behind it, not a DNS edit. Every staging name is free.

Everything created in AWS is named `examprep-`, so a cost report or a cleanup is a filter rather than archaeology. Two of those names can **never** be changed once created — **a security group's and an IAM role's** — so the ten minutes before the first `create-security-group` is the cheap moment to decide. A VPC's name is only a tag. An S3 bucket name is globally unique across every AWS account, which is the one you cannot quietly reclaim later.

Two things deliberately keep the old name: the metric prefix is `iace_` in `apps/api/src/common/metrics/metrics.service.ts`, and renaming it orphans every dashboard; and the compose project names `iace-api` and `iace-valkey` namespace the Docker volumes, so changing one strands the volume holding Caddy's certificates.

3.4 Today's shortcut, and it is the right one

Custom subdomains for the two websites need a certificate from ACM in **us-east-1**, validation records at GoDaddy and a CloudFront change — about 40 minutes, mostly waiting. CloudFront hands out a working `https://d111111abcdef8.cloudfront.net` with a valid certificate the moment you create it. **Launch on those**, and add the custom names later with §12.6.

The API subdomain is not optional. Caddy cannot get a certificate without it, and the student app is an installable PWA whose service worker will not register without real HTTPS.

---

4. AWS groundwork

4.1 The VPC and its subnets

Console → **VPC** → **Create VPC**, choose **VPC and more**, which builds the whole set in one step.

Field	Value	Why
Name tag	`examprep`	Everything it creates gets this prefix
IPv4 CIDR	`10.0.0.0/16`	65,536 addresses; you will use a dozen
IPv6	No	Nothing needs it
Availability Zones	**2**	RDS refuses a subnet group with fewer
Public subnets	**2**	Boxes A and B live in the first
Private subnets	**2**	RDS lives here
NAT gateways	**None**	See below — a $32/month decision
VPC endpoints	**None**	
DNS hostnames / resolution	Both enabled	RDS hands you a hostname, not an address

Why no NAT gateway. A NAT lets machines in a private subnet reach the internet, at about $32 a month plus traffic. Both your boxes sit in a *public* subnet with an address of their own, so they reach Docker Hub, Grafana and the SMS provider directly. The firewall work is done by security groups instead — which is where it belongs, because a group travels with the machine and a subnet does not.

The Name tag does not follow a rename. The wizard copies it onto every subnet, route table and gateway *at creation*, as separate tags. Rename the VPC later and the children keep the old prefix — they have to be retagged one by one.

What it produced here, for shape:

VPC	`vpc-0d2c4a69d1554580d`
Public 1a / 1b	`subnet-04e8d9f03bd4882bd` `10.0.0.0/20` · `subnet-059d22aac57f827f6` `10.0.16.0/20`
Private 1a / 1b	`subnet-0b71839277c770eed` `10.0.128.0/20` · `subnet-01c4d9996329c3371` `10.0.144.0/20`
Internet gateway	`igw-02b232c3417ac5225`

Check the routing rather than trusting the names: the public route table must carry `0.0.0.0/0 → igw-…` and the private ones must not.

```
aws ec2 describe-route-tables --filters Name=vpc-id,Values=<vpc-id> \
  --query 'RouteTables[].{Name:Tags[?Key==`Name`]|[0].Value,Routes:Routes[].GatewayId}' --output
table
```

4.2 Two security groups — before any instance exists

Console → **VPC** → **Security groups** → **Create security group**, twice, **in this order**: the second one's rules name the first as their source, so the first must exist. It only has to exist — its own rules can come later.

Set the VPC to `examprep` on both. The dropdown defaults to your *default* VPC, and a group belongs to exactly one VPC, chosen at creation and **never changeable** — one in the wrong VPC simply will not appear when you launch the instance.

Do this before launching anything. With no group to pick, the launch wizard creates `launch-wizard-1` with SSH open to the whole internet — the exact rule this design avoids. An instance's groups *can* be changed afterwards; the group's own name and VPC cannot.

`examprep-api` — Box A.

Direction	Type	Port	Source	Why
Inbound	HTTP	80	`0.0.0.0/0`	The ACME challenge arrives here, and Caddy redirects the rest to 443
Inbound	HTTPS	443	`0.0.0.0/0`	The actual traffic
Outbound	All	All	`0.0.0.0/0`	The default; leave it

There is no SSH rule, deliberately. A rule locked to "my IP" holds until that address rotates, and what happens then — at eleven at night, with something broken — is that somebody widens it to the whole internet and never narrows it again. You shell in through SSM Session Manager instead: no inbound rule, no key file, and a transcript.

`examprep-data` — Box B and RDS.

Direction	Type	Port	Source
Inbound	Custom TCP	6379	**sg** `examprep-api`
Inbound	Custom TCP	6380	**sg** `examprep-api`
Inbound	PostgreSQL	5432	**sg** `examprep-api`

In the Source box type `sg-` and pick `examprep-api` — never an address range. That is the whole point: the rule names a *group*, so any box joining it is trusted and you never touch this rule again. Only groups in the same VPC appear in that dropdown, which is where a VPC mix-up shows itself first.

4.3 The instance profile

IAM → Roles → Create role, trusted entity **AWS service**, use case **EC2**.

This takes two passes, and the wizard does not say so. Step 2's "Use existing policy" and "Create inline policy" are a radio pair — pick one and the other's panel disappears. Nothing is being overwritten; you simply cannot do both before the role exists.

Pass one: leave *Use existing policy* selected and tick `AmazonSSMManagedInstanceCore` (this is what makes Session Manager work) and `CloudWatchAgentServerPolicy`. A ticked policy stays ticked when you change the search, and the count at the top proves it. Name the role `examprep-box`, create it.

Pass two: on the role → Permissions → Add permissions → **Create inline policy** → **JSON**. Paste the whole document below as **one** policy named `examprep-box-inline` — all four statements together, not four policies. Inline policies are keyed by name, so a second sharing a name replaces the first with no warning. Stay on the JSON tab; switching to Visual and back can rewrite what you pasted.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "MediaBucket",
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject", "s3:DeleteObject", "s3:ListBucket"],
      "Resource": [
        "arn:aws:s3:::examprep-staging-media",
        "arn:aws:s3:::examprep-staging-media/*"
```

```
      ]
    },
    {
      "Sid": "ReadItsOwnEnvironment",
      "Effect": "Allow",
      "Action": ["ssm:GetParameter"],
      "Resource": "arn:aws:ssm:ap-south-1:<ACCOUNT_ID>:parameter/examprep/*"
    },
    {
      "Sid": "DecryptIt",
      "Effect": "Allow",
      "Action": ["kms:Decrypt"],
      "Resource": "*",
      "Condition": {
        "StringEquals": { "kms:ViaService": "ssm.ap-south-1.amazonaws.com" }
      }
    },
    {
      "Sid": "PullImages",
      "Effect": "Allow",
      "Action": [
        "ecr:GetAuthorizationToken",
        "ecr:BatchGetImage",
        "ecr:GetDownloadUrlForLayer",
        "ecr:BatchCheckLayerAvailability"
      ],
      "Resource": "*"
    }
  ]
}
```

`<ACCOUNT_ID>` is your twelve digits — `aws sts get-caller-identity` prints them. Left as-is it is not a valid ARN and IAM rejects it.

Note what is NOT in it: `ssm:PutParameter`. A box reads its environment and cannot overwrite it, so an attacker on the box cannot poison what every replacement pulls afterwards. Pushing is an admin action from your laptop (§6.4), and `put-parameter` run on a box is *supposed* to be denied.

Check what actually landed, because the console view is easy to misread:

```
aws iam list-attached-role-policies --role-name examprep-box
aws iam list-role-policies --role-name examprep-box
aws iam get-role-policy --role-name examprep-box --policy-name examprep-box-inline
```

4.4 An Elastic IP, and encryption by default

EC2 → Elastic IPs → Allocate. Do not associate it yet. It costs $3.65 a month attached or not, and it is what makes a replacement box a three-minute fix.

Then, once, before you launch anything:

```
aws ec2 enable-ebs-encryption-by-default --region ap-south-1
```

Every volume created afterwards is encrypted with the free AWS-managed key. **Like RDS encryption, it cannot be switched on in place later** — the fix is snapshot, copy with encryption, swap. One command now, or twenty minutes per box later.

---

5. RDS — the database

Create this early and leave it building; it takes 10–15 minutes and nothing waits on it.

Console → **RDS** → **Create database** → **Standard create**.

Field	Value	Why
Engine	PostgreSQL	
Version	**the highest 17.x**	Not 18 — see below
Template	**Dev/Test** (or Free tier)	Only a form pre-fill; see below
Identifier	`examprep`	One instance, a database per environment
Master username	`postgres`	
Credentials management	**Self managed**	See below
Master password	`openssl rand -hex 24`	**Not** the password the app uses
Instance class	**db.t4g.micro**	Not `db.t3.micro` — same machine, $18.98 against $15.33
Storage	gp3, **20 GB**, autoscaling to 100	3,000 IOPS at any size
Multi-AZ	**No**	See §17 — for production this is an open decision
Compute resource	**Don't connect to an EC2 compute resource**	See below
VPC	`examprep`	
Subnet group	**Create new** — `examprep-db-private`, the two **private** subnets only	Never "Automatic setup"
Public access	**No**	
Security group	`examprep-data`	Remove `default` if pre-selected
Availability Zone	`ap-south-1a`	Same zone as the boxes: free and fast
Database authentication	**Password**	Not IAM — see below
Initial database name	`examprep_staging`	Blank here creates no database at all
Backup retention	**7 days**	Point-in-time restore, free up to allocated storage
Backup window	`19:00–19:30 UTC`	00:30 IST, nobody is sitting an exam
Auto minor version upgrade	**On**	Security fixes land without anyone remembering
Maintenance window	`Sun 20:00–20:30 UTC`	That upgrade is a brief restart; RDS refuses an overlap with backups, and the default is AWS's guess
Encryption	**On**, default key	Free, and **cannot be enabled later**
Performance Insights	On, 7 days	Free at that retention
Deletion protection	**On**	

The Template radio is a form pre-fill and nothing else — not stored, not visible afterwards, no part of the instance. "Production" turns Multi-AZ on and switches storage to Provisioned IOPS with a 100 GB and 1,000 IOPS floor, which is the quietest way to turn an $18 database into a three-figure one. The fields in this table are what matter.

Why 17 and not 18. The reason is in the repo: `docker-compose.yml` pins `postgres:17-alpine`, so every test and every migration has run against 17, and Prisma 6.19.3 is validated there. On 18, staging becomes the only place 18 is ever exercised and the first difference surfaces in a demo rather than in CI. 17 runs to **28 February 2030**; after a version leaves standard support, RDS Extended Support costs $0.114 per vCPU-hour, more than the instance. Take the highest **17.x** — `17.11-R1` means community 17.11, AWS build 1; inside the database `select version()` says 17.11.

`t4g` rather than `t3`, and on RDS it is free money. Same vCPU, same memory, same burstable credit model for 19–21% less. On EC2, Graviton costs something: the AMI and images must be ARM. A database has no such cost, because nothing of yours runs on it — Prisma and `psql` speak the wire protocol and cannot tell what the server was compiled for.

Self managed credentials. The alternative hands the master password to Secrets Manager at $0.40 a month, and what it manages is the **master** credential only — not the application's, which is what people assume. The app connects as `examprep_staging` with a password in `deploy/.env`. So the managed thing would be a credential used about three times ever. Put it in the institute's password manager and in SSM, free, the same mechanism as §11. Reversible either way.

Password authentication, not IAM. IAM auth is nicer in principle — short-lived tokens instead of a password — but Prisma takes one static URL and has no hook for a per-connection credential. A 15-minute token works until the pool recycles a connection, so the failure arrives an hour in, intermittently. Additive and reversible later if Prisma grows the support.

Don't connect it to an EC2 compute resource. Box A may not exist yet, but the real reason is that the shortcut creates its *own* pair of security groups and attaches them to both the database and your instance — leaving that pair beside `examprep-api` / `examprep-data`, two mechanisms doing one job, and no obvious answer later to which rule is load-bearing. What §4.2 built is strictly better: the rule names a *group*.

Never let it create the subnet group automatically. Automatic setup puts *every* subnet in the VPC into the group, public ones included, and RDS then places the instance — and any replacement or Multi-AZ standby — anywhere in that set. `Public access: No` still means no public address, but the database would sit in a subnet with a route to the internet gateway. Tick both availability zones first: the subnet list only shows subnets in the zones you selected.

5.1 What you cannot change afterwards

Fixed at creation	Changeable later
The VPC	Instance class — brief outage
The master **username**	Storage size, up only and online
The initial **database name**	Storage type
Encryption at rest	Subnet group — *same VPC only*, disruptive
	Security groups — immediate, no outage
	Multi-AZ on or off
	Public accessibility
	Backup retention and both windows

Fixed at creation	Changeable later
	Master password, and who manages it
	Minor and major engine version
	Deletion protection
	The identifier, though the endpoint changes with it

If the VPC is wrong, delete and start again — with no data that is fifteen minutes, not a migration: Modify → uncheck Deletion protection → Apply immediately, then Actions → Delete with no final snapshot.

5.2 The extension, then the role

This runs from **Box A** — RDS has no public address and only `examprep-api` can reach it — so come back here after §7.3. Let psql prompt rather than putting the password in a URI: a password from `openssl rand -base64` contains `/`, which ends the password field and silently truncates it.

```
psql -h examprep.ct04iak44pyz.ap-south-1.rds.amazonaws.com -U postgres -d examprep_staging -W
```

```
-- As MASTER, before any migration runs. 20260925120000 calls CREATE EXTENSION, and on RDS that
-- needs rds_superuser, which the application role deliberately will not have. Created here, the
-- migration's IF NOT EXISTS is a no-op. Skip it and `prisma migrate deploy` dies on a permission
-- error — and the API containers wait on migrate exiting zero, so nothing starts at all.
CREATE EXTENSION IF NOT EXISTS pg_trgm;

-- 85, NOT 25. See below: these are two different numbers.
CREATE USER examprep_staging WITH PASSWORD 'PASTE_A_HEX_PASSWORD' CONNECTION LIMIT 85;

-- Postgres grants CONNECT to PUBLIC on every new database, so without this any role on the
-- instance can open a connection here — including production's, the day it exists.
REVOKE CONNECT ON DATABASE examprep_staging FROM PUBLIC;
GRANT ALL PRIVILEGES ON DATABASE examprep_staging TO examprep_staging;

-- Not optional on Postgres 15+: the public schema stopped granting CREATE to everyone, so without
-- this Prisma cannot create a single table.
GRANT ALL ON SCHEMA public TO examprep_staging;

ALTER ROLE examprep_staging SET statement_timeout = '30s';
ALTER ROLE examprep_staging SET idle_in_transaction_session_timeout = '60s';
```

```
\dx
\du examprep_staging
\q
```

`CONNECTION LIMIT` **and** `connection_limit` **are different numbers wearing one name.** `connection_limit=25` in `DATABASE_URL` is Prisma's pool **per container**; `CONNECTION LIMIT` on the role is a cap **across every container using it**. Three containers share one URL and can grow to 75 together, so a role capped at 25 refuses the second and third with `FATAL: too many connections for role` — **the moment load arrives, not at boot**, which is what makes it nasty. 85 is 75 plus the short-lived `migrate` container and a `psql` session or two.

Where 25 itself comes from: `QUEUE_POLICY` in `apps/api/src/queue/queues.ts` sums to exactly **20** concurrent slots (scoring 8, rollup 2, notifications 4, delivery 2, four singletons), so a worker has twenty jobs wanting a connection before it serves a request. Prisma's default pool is `cpus × 2 + 1`, which is five.

Generate the password with `openssl rand -hex 24`, not base64 — **hex has no `/`, `+` or `=`** to break a connection string. Prove the role before going further:

```
PGPASSWORD='the-hex' psql -h examprep.ct04iak44pyz.ap-south-1.rds.amazonaws.com \
  -U examprep_staging -d examprep_staging -c "select current_user, current_database();"
```

---

6. Box B — Valkey

6.1 Launch it

Console → **EC2** → **Launch instance**. You made the security groups in the *VPC* console; instances are in the *EC2* one.

Field	Value
Name	`examprep-valkey`
AMI	**Amazon Linux 2023**, architecture **64-bit (Arm)**
Instance type	**t4g.micro**
Key pair	**Proceed without a key pair**
Network settings → Edit → VPC	`examprep`
Subnet	the **public** subnet in `ap-south-1a`
Auto-assign public IP	**Enable**
Firewall	Select existing → `examprep-data`
Storage	10 GiB gp3
Advanced → IAM instance profile	`examprep-box`
Advanced → Metadata version	**V2 only (token required)**
Advanced → Metadata response hop limit	**2**

The architecture is the trap. `t4g` is Graviton, which is ARM. Pick an x86 AMI and the instance will not boot, with an error that does not say why. Set the architecture selector before choosing the image.

Auto-assign public IP may default to Disabled — the VPC wizard does not always set `MapPublicIpOnLaunch` on its public subnets. Without an address this box has no route to the internet at all, and `dnf install docker` simply hangs with nothing suggesting a network cause.

Hop limit 2, not the default 1. The metadata service hands out your role's credentials at `169.254.169.254`. IMDSv2 requires a PUT for a token first, which blocks the one-request SSRF that would otherwise walk off with the role. The hop limit is how many network hops the response may travel: 1 works from the host, but a process inside a container is one hop further out and gets nothing. At 1 your containers silently cannot reach S3 or SSM.

Why it has a public address at all: only to pull the Valkey image. Nothing on the internet has a rule that reaches it.

6.2 A shell, and Docker

```
aws ssm start-session --target i-02d91009dacaa7177
```

If it says the target is not connected, wait two minutes — the agent registers shortly after boot — and confirm the instance profile is attached.

```
sudo dnf update -y
sudo dnf install -y docker git
sudo systemctl enable --now docker
sudo usermod -aG docker ssm-user
```

```
exit
```

`exit` ends the session; that is what you want. Start a new one so the group applies, then check **both** halves:

```
docker --version && docker compose version
```

The second will fail, and the next step needs it. Amazon Linux 2023's `docker` package does not ship the Compose v2 plugin and it is not in the repositories either:

```
sudo mkdir -p /usr/local/lib/docker/cli-plugins
sudo curl -sSL https://github.com/docker/compose/releases/latest/download/docker-compose-linux-aarch64 -o /usr/local/lib/docker/cli-plugins/docker-compose
sudo chmod +x /usr/local/lib/docker/cli-plugins/docker-compose
docker compose version
```

`aarch64`**, not** `x86_64`**.** The x86 binary downloads perfectly and then fails with "cannot execute binary file", which reads like a corrupt download rather than the wrong architecture.

Box A needs `buildx` **as well**, and it is the same gap — Amazon Linux ships neither plugin. Compose refuses with `compose build requires buildx 0.17.0 or later`, and `apps/api/Dockerfile` needs BuildKit anyway for `RUN --mount=type=cache`, so `DOCKER_BUILDKIT=0` is not a way out: the build would then fail on the Dockerfile instead. Unlike Compose, the buildx asset name carries its version, so there is no unversioned `latest/download` path:

```
V=$(curl -s https://api.github.com/repos/docker/buildx/releases/latest | grep -o '"tag_name": *"[^"]*"' | cut -d'"' -f4)
sudo curl -sSL "https://github.com/docker/buildx/releases/download/$V/buildx-$V.linux-arm64" -o /usr/local/lib/docker/cli-plugins/docker-buildx
sudo chmod +x /usr/local/lib/docker/cli-plugins/docker-buildx
docker buildx version
```

Box B does not build anything, so it needs only Compose.

6.3 Valkey

Do not clone the repository on this box. It needs exactly two files, and cloning would put a credential on a machine whose only job is one container. Box A is different (§7.3) because it builds from source.

```
sudo mkdir -p /opt/examprep/deploy/valkey
sudo chown -R ssm-user:ssm-user /opt/examprep
cd /opt/examprep/deploy/valkey
```

Own the directory first, then use a plain redirect. `sudo cat > file` does not work: the `>` is handled by your shell before `sudo` runs, so the file is opened as you regardless and you get permission denied on a line that looks like it should have worked. Where a path is not yours, the alternative is `sudo tee file > /dev/null <<'EOF'`, where `tee` itself runs privileged.

```
cat > compose.yml <<'YAML'
name: iace-valkey

services:
  valkey-prod:
    image: valkey/valkey:8-alpine
    restart: unless-stopped
    command:
      - valkey-server
```

```
      - --appendonly
      - 'yes'
      - --maxmemory-policy
      - noeviction
      - --maxmemory
      - ${PROD_MAXMEMORY:-1200mb}
      - --requirepass
      - ${VALKEY_PROD_PASSWORD:?set it in .env}
    environment:
      REDISCLI_AUTH: ${VALKEY_PROD_PASSWORD}
    ports:
      - '${PRIVATE_IP}:6379:6379'
    volumes:
      - prod-data:/data
    stop_grace_period: 60s
    healthcheck: &ping
      test: ['CMD', 'valkey-cli', 'ping']
      interval: 10s
      timeout: 5s
      retries: 10

  valkey-staging:
    image: valkey/valkey:8-alpine
    restart: unless-stopped
    command:
      - valkey-server
      - --appendonly
      - 'yes'
      - --maxmemory-policy
      - noeviction
      - --maxmemory
      - ${STAGING_MAXMEMORY:-256mb}
      - --requirepass
      - ${VALKEY_STAGING_PASSWORD:?set it in .env}
    environment:
      REDISCLI_AUTH: ${VALKEY_STAGING_PASSWORD}
    ports:
      - '${PRIVATE_IP}:6380:6379'
    volumes:
      - staging-data:/data
    stop_grace_period: 60s
    healthcheck: *ping

volumes:
  prod-data:
  staging-data:
YAML
```

Keep `name: iace-valkey` **exactly as it is** — it namespaces the volumes holding the append-only file, so changing it strands the data. `deploy/valkey/compose.yml` in the repository is the source of record if this drifts; it is reproduced here only because this box has no checkout.

`REDISCLI_AUTH` is why the healthcheck works: `valkey-cli ping` answers `NOAUTH` the moment `requirepass` is set, so without it every container reports unhealthy for ever. In the environment rather than on the command line, so the password stays out of `ps`.

This box's **private** address — the `10.0.x.x` one, not the public:

```
TOKEN=$(curl -sX PUT "http://169.254.169.254/latest/api/token" -H "X-aws-ec2-metadata-token-ttl-
seconds: 60")
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-data/local-ipv4;
```

```
echo
```

Two passwords, as two separate runs. Both are needed even though only staging runs today — the `:?` guard refuses an empty one rather than starting a Valkey with no password.

```
openssl rand -base64 36
```

```
cat > .env <<'ENV'
PRIVATE_IP=10.0.0.32
VALKEY_PROD_PASSWORD=
VALKEY_STAGING_PASSWORD=
PROD_MAXMEMORY=1200mb
STAGING_MAXMEMORY=256mb
ENV
vi .env
```

```
sudo chown root:root .env && sudo chmod 600 .env
sudo docker compose up -d valkey-staging
sudo docker compose ps
```

Everything from here needs `sudo`, including `vi`. Opening a `root:root 600` file without it gives a read-only buffer, and vi only says so when you try to save.

Three locks, and the third is the one that survives a mistake. The security group; `PRIVATE_IP` publishing the port on the VPC address rather than every interface; and `requirepass`. Behind them is every live sitting, every session token and every OTP, on among the most-scanned ports there is. Prove the second and third rather than assuming them:

```
sudo ss -ltnp | grep 638
sudo docker compose exec -T valkey-staging valkey-cli config get maxmemory-policy
sudo docker run --rm --network host valkey/valkey:8-alpine valkey-cli -h 10.0.0.32 -p 6380 ping
```

The listener must be on `10.0.0.32:6380` and not `0.0.0.0`, the policy must read `noeviction`, and that last ping must answer `NOAUTH Authentication required`. A `PONG` there means no password and the stack came up anyway.

6.4 Push the file to SSM — from your laptop, not the box

The instance profile carries `ssm:GetParameter` and deliberately not `ssm:PutParameter` (§4.3), so `put-parameter` on the box is denied by design: **a box pulls its secrets and must not be able to overwrite them.** On your laptop, with the two passwords:

```
cat > /tmp/valkey.env <<'ENV'
PRIVATE_IP=10.0.0.32
VALKEY_PROD_PASSWORD=
VALKEY_STAGING_PASSWORD=
PROD_MAXMEMORY=1200mb
STAGING_MAXMEMORY=256mb
ENV
vi /tmp/valkey.env
aws ssm put-parameter --region ap-south-1 --name /examprep/valkey/env \
  --type SecureString --overwrite --value file:///tmp/valkey.env
rm /tmp/valkey.env
```

The box's own `GetParameter` brings it back on a replacement, which is the half that has to work under pressure.

---

7. Box A — the API box

7.1 Launch it

As §6.1, with these differences:

Field	Value	Why
Name	`examprep-api-staging`	
Instance type	**t4g.medium** (2 vCPU, 4 GB)	While you build on the box — see below
Security group	`examprep-api`	Not `examprep-data`
Storage	**20 GiB** gp3	See §7.5

Everything else identical: ARM AMI, no key pair, the public subnet in `1a`, auto-assign public IP **Enable**, profile `examprep-box`, IMDSv2 required, hop limit **2**.

Then **EC2 → Elastic IPs** → your address → Actions → Associate → this instance.

Nothing of the front end is built here. The image copies only `apps/api`, `packages` and `prisma`, and runs `pnpm --filter @iace/api... build` — the API plus `@iace/contracts` and `@iace/config`. Neither SPA is in the image. The Vite builds happen wherever `deploy/publish-spas.sh` runs, which is your laptop or CI.

What wants the 4 GB is `pnpm install`, and it installs more than the API needs: `apps/api/Dockerfile` copies **every** workspace manifest before installing so the lockfile stays frozen, and the side effect is resolving Expo, React Native and both SPA toolchains. Add Prisma's engine downloads and `tsc`.

Drop to `t4g.small` once images come from ECR — not `micro`. The three containers need ~850 MB at rest, plus Caddy, Alloy and the OS: about 1.4 GB, which does not fit in micro's 1 GB. Same two vCPU either way; the difference is memory, and the saving is $4.09 a month against an OOM kill mid-sitting.

7.2 Shell and tooling

As §6.2 — Docker, the Compose plugin by hand with `aarch64` — plus the Postgres client, because §5.2 runs from here:

```
sudo dnf install -y postgresql17
```

7.3 The repository

Box A **does** clone, because it builds from source and the compose file, Caddyfile and Alloy config all live in the tree.

```
sudo mkdir -p /opt/examprep && sudo chown -R ssm-user:ssm-user /opt/examprep
cd /opt/examprep
git clone https://github.com/Indian-Academy-of-Competitive-Exams/iace-exam-portal.git .
git switch staging
```

`git switch staging` **is not optional.** A clone lands on `main`, and `main` is production. Work lands on `staging` and that is the branch this box should track.

While the repository is **public** the clone needs no credential. The day it goes private, this box needs a read-only deploy key and the remote switched to SSH — generate on the box, paste the `.pub` into the repository's Settings → Deploy keys.

`git` **run over SSM says "detected dubious ownership" and prints nothing useful.** SSM runs commands as root and the checkout belongs to `ssm-user`, so git refuses — to stderr, which makes a blank answer look like an empty repository rather than a refusal. Use `sudo -u ssm-user git -C /opt/examprep …`.

7.4 The environment file

```
sudo cp deploy/.env.example deploy/.env
```

Four secrets, as four separate runs — do **not** put the explanation after the command as a `#` comment: zsh does not treat `#` as a comment in an interactive shell unless `interactive_comments` is set, so a pasted line arrives at the program with the comment attached and fails with something unhelpful like `rand: Extra option`.

```
openssl rand -base64 48
```

```
sudo vi deploy/.env
```

Variable	Value	Notes
`API_HOST`	`api.staging.examprep.iace.co.in`	Must match §8 exactly, or no certificate
`ACME_EMAIL`	`developer@iace.co.in`	Where Let's Encrypt sends expiry warnings
`NODE_ENV`	`development`	**Deliberate** — see below
`OTP_SENDER`	`console`	The code comes back in the response as `devCode`
`DATABASE_URL`	`postgresql://examprep_staging:HEX@examprep.ct04iak44pyz.ap-south-1.rds.amazonaws.com:5432/examprep_staging?schema=public&connection_limit=25&pool_timeout=20`	`connection_limit` is not optional
`REDIS_URL`	`redis://:VALKEY_STAGING_PASSWORD@10.0.0.32:6380`	Box B's **private** address. The empty username before the colon is required
`JWT_*`, `PIN_PEPPER`, `METRICS_TOKEN`	the four generated values	
`S3_BUCKET`	`examprep-staging-media`	
`S3_ACCESS_KEY_ID` / `SECRET`	**leave empty**	The instance profile signs; the API refuses half a pair
`MEDIA_BASE_URL`	a placeholder until §11	**Cannot be blank** — see below

Variable	Value	Notes
`CORS_ORIGINS`	empty until §11	Empty reflects any origin, which is what lets a local SPA talk to this box
`TRUST_PROXY_HOPS`	`1`	Caddy is the one hop. At 0 every caller shares one rate-limit bucket and the hall locks itself out
`ENVIRONMENT`	`staging`	The `env` label on every log line
`LOKI_*` / `PROM_*`	§14	Blank is fine; Alloy just fails to ship

`MEDIA_BASE_URL` **cannot be blank.** It is `.min(1, 'MEDIA_BASE_URL is required')` with no default, so the API refuses to boot without it. Any non-empty string passes until §11 gives you the real CloudFront domain.

And no bucket name in it when you do. The API joins base and key as `${MEDIA_BASE_URL}/${key}` and the media distribution's origin *is* the bucket, so the key alone is the path. Put the bucket in as well and CloudFront asks S3 for `examprep-staging-media/questions/images/…`, which does not exist — **every question image a 404**. The local `.env.example` does carry the bucket, because MinIO is path-style; CloudFront is not.

Why `NODE_ENV=development` **on a deployed box.** In production mode the API refuses to start without a real SMS path — a student signs in with an SMS OTP and an admin with an emailed one, and the DLT registration behind that SMS does not exist yet (§20). In development, `OTP_SENDER=console` returns the code in the HTTP response as `devCode`, so a tester signs in with no provider at all. `CORS_ORIGINS` is enforced either way.

```
sudo chown root:root deploy/.env && sudo chmod 600 deploy/.env
```

Back it up **from your laptop**, for the §6.4 reason. Keep the filled-in file locally, strip the comments, push that — a Standard-tier parameter caps at **4 KB** and the commented file is close:

```
grep -v '^#' env.staging | grep . > /tmp/env.stripped
aws ssm put-parameter --region ap-south-1 --name /examprep/staging/env \
  --type SecureString --overwrite --value file:///tmp/env.stripped
rm /tmp/env.stripped
```

7.5 Why 20 GB, and how it stays there

Logs are not what fills the disk: six containers capped at 10 MB × 3 is **180 MB**. Docker is — every release leaves the image it replaced behind, and a build leaves layers and cache nothing collects on its own. Once images come from ECR the box holds the OS and Docker (~4 GB), two releases (~4 GB) and volumes and logs (under 1 GB): about 9 GB of 20.

```
sudo ./deploy/prune-disk.sh
```

Keeps the newest **two** builds of each app image and drops the rest, plus stopped containers, build cache and dead networks. A count rather than an age because early on a release goes out several times an afternoon: seven days of retention bounds nothing — twenty-one image pairs at ~1.3 GB fills 20 GB in three days — and what breaks then is not the prune but the next release, possibly at the `migrate` step with the API already stopped.

Two and not one because Docker shares layers: a code-only release differs by one application layer, so the spare costs a few hundred megabytes and buys a rollback needing no network — which matters precisely when the reason for rolling back is that the box cannot reach ECR. Nothing is lost either way: **the images on the box are a cache, the archive is ECR**, whose lifecycle keeps ten.

Never add `--volumes` to a prune. They hold Caddy's issued certificates and Box B's append-only file, and Let's Encrypt rate-limits five certificates per hostname per week — one careless `docker system prune --volumes` locks you out of your own domain for seven days.

While the box still builds, prune after each build. gp3 also grows online, so 20 → 30 is a Modify and `xfs_growfs /`, not a rebuild.

---

8. DNS at GoDaddy

GoDaddy → **My Products → Domains → iace.co.in → DNS → Manage Zones → Add New Record**.

8.1 The one record you must have

Field	Value
Type	`A`
Name	`api.staging.examprep`
Value	your Elastic IP, e.g. `3.109.77.9`
TTL	Custom → **600**

The Name field is relative to the zone. You own `iace.co.in`, so `api.staging.examprep` creates `api.staging.examprep.iace.co.in`. Typing the full name creates `…iace.co.in.iace.co.in`, which is the single most common mistake here and shows up as a Caddy certificate failure rather than a DNS error.

Leave the existing MX, TXT and `www` records alone — the institute's mail and main website.

Check before starting Caddy, and check the *authoritative* server rather than your own resolver, which caches the old negative answer:

```
dig +short api.staging.examprep.iace.co.in
dig +short @ns69.domaincontrol.com api.staging.examprep.iace.co.in
dig +short @8.8.8.8 api.staging.examprep.iace.co.in
```

Your laptop may lag by minutes while GoDaddy and Google already answer. **Let's Encrypt queries public resolvers, not yours**, so those two are what matter. It rate-limits **five failed validations per hostname per hour**, which is the one thing that can cost you an hour.

8.2 The websites get no Elastic IP and no A record — ever

The API box has an address of its own, so it takes an **A** record. The websites are not on a box: they are files in S3 served by CloudFront, which is a few hundred edge locations. DNS hands each visitor the addresses nearest them — a different answer in Mumbai and Delhi — and those change without notice. An A record would pin every student to one edge that may not exist next month.

	Backend	Frontend
What runs it	One EC2 box	CloudFront edges worldwide
Address	An Elastic IP, yours, fixed	None you can see or pin
DNS record	`A` → `13.234.56.78`	`CNAME` → `d111111abcdef8.cloudfront.net`
Certificate	Caddy, from Let's Encrypt	ACM in us-east-1, free
Where TLS ends	Caddy, on your box	At the edge nearest the student

Which is also why the websites need no DNS today: each distribution arrives with `https://d111111abcdef8.cloudfront.net` and a valid certificate already on it.

8.3 The website records — after §11, optional

Type	Name	Value	TTL
CNAME	`staging.examprep`	`d111111abcdef8.cloudfront.net`	600

Type	Name	Value	TTL
CNAME	`admin.staging.examprep`	`d222222abcdef8.cloudfront.net`	600

These alone are **not enough**, and the failure is confusing: a distribution rejects any `Host` it has not been told about and answers `403 The request could not be satisfied`, which reads like a permissions problem. §12.6 is the other half.

You cannot do this for the bare domain. DNS forbids a CNAME at a zone apex and GoDaddy's free DNS has no ALIAS. If `iace.co.in` itself ever has to point at CloudFront, that is when the zone moves to Route 53, whose ALIAS record does exactly this and is free.

---

9. Bring the stack up

```
cd /opt/examprep
sudo docker compose -f deploy/compose.yml --env-file deploy/.env up -d --build
```

The first build takes **8–15 minutes** on a `t4g.medium`.

Service	Expected
`migrate`	`Exited (0)` — it ran the migrations and stopped. Correct, not a failure
`exam`, `core`, `worker`	`Up … (healthy)`
`caddy`, `alloy`	`Up`

The order is enforced, not hoped for. The three API containers declare `depends_on: migrate: condition: service_completed_successfully`, so Docker will not start them until migrate has exited zero. A release can never serve against an unmigrated schema.

9.1 Without DNS, if you are waiting on it

Everything except TLS can be done first. Set `API_HOST=:80` and Caddy serves plain HTTP with no certificate at all — **it is the hostname, not the port, that triggers automatic HTTPS**. A site address of `:80` has no name to request a certificate for, so Let's Encrypt is never contacted and the rate limit is never spent, while the real routing still runs.

Beware `:443`: no hostname but a TLS port makes Caddy issue itself a **self-signed** certificate from its internal CA, which every browser rejects. It looks like the secure choice and is not.

Narrow port 80 to your own address while you do this, then put `0.0.0.0/0` back before you try the certificate — the ACME challenge comes from Let's Encrypt's servers, not from you.

```
MYIP=$(curl -s https://checkip.amazonaws.com)
aws ec2 revoke-security-group-ingress --group-id sg-0a97263d7583a4359 --protocol tcp --port 80 --
cidr 0.0.0.0/0
aws ec2 authorize-security-group-ingress --group-id sg-0a97263d7583a4359 --protocol tcp --port 80 -
-cidr "${MYIP}/32"
```

9.2 Watch the certificate

```
sudo docker compose -f deploy/compose.yml --env-file deploy/.env logs caddy | grep -i certificate
curl -i https://api.staging.examprep.iace.co.in/health
```

A JSON body and `HTTP/2 200` means the whole chain works: DNS, the security group, Caddy, TLS and the `core` container behind it.

If the certificate does not issue, in order of likelihood: DNS has not propagated; port 80 is not open to the world; or `API_HOST` does not exactly match the record.

9.3 Two health endpoints, and why

```
curl -s https://api.staging.examprep.iace.co.in/health | jq
curl -s https://api.staging.examprep.iace.co.in/health/ready | jq
```

`/health` answers 200 while the process is alive, even when a dependency is down. `/health/ready` probes Postgres, Valkey, S3 and the queue — and is what the container healthcheck uses.

S3 is probed but does not gate readiness. `READINESS_DEPENDENCIES` is `['database', 'redis', 'queue']`, so a bucket that does not exist yet shows as `down` in the body while the container still reports healthy. The stack comes up before §11.

---

10. Seed the first super admin

Migrations create tables and no rows. Admins cannot self-register and nothing in the application creates one, so **until you do this there is no way into the admin app at all.**

```
sudo docker compose -f deploy/compose.yml --env-file deploy/.env run --rm \
  --entrypoint sh migrate -c 'npx prisma db execute --file prisma/seed.sql --schema
prisma/schema.prisma'
```

That inserts the super admin `developer@iace.co.in`, the eleven branches, the SSC CGL exam with its two tiers, and the Tier 1 base configuration with its four sections. Idempotent — every insert is guarded.

The ONLINE branch is not optional. A student's branch decides which standard series they reach, so a student without one sees only what a free series, a program, an event or an explicit grant opens. The API refuses to duplicate, rename, retire or delete it.

```
curl -s -X POST https://api.staging.examprep.iace.co.in/auth/admin/otp/request \
  -H 'content-type: application/json' -H 'x-client: WEB' \
  -d '{"email":"developer@iace.co.in"}' | jq
```

Because `OTP_SENDER=console` the response carries `devCode`. That is the whole point of running staging in development mode: no email provider, no SMS vendor, and a tester can still sign in.

---

11. The websites — S3 and CloudFront

Both single-page apps are files. There is no server to run, and **none of this needs DNS**.

Six things happen here, in this order, and the order matters: two buckets, three distributions, a bucket policy for each bucket, one CloudFront function, the API's environment, then the first publish. The bucket policies cannot be written until the distributions exist, because a policy names a distribution by ARN.

11.1 Two buckets

S3 → **Buckets** → **Create bucket**. Twice.

Field	Value
Bucket type	General purpose
Bucket name	`examprep-staging-spas`, then `examprep-staging-media`
Region	Asia Pacific (Mumbai) `ap-south-1`
Object Ownership	ACLs disabled
Block Public Access	**all four boxes ticked**
Bucket Versioning	Disable
Default encryption	SSE-S3, Bucket Key enabled

Every row except the name is already the console default, so in practice you type a name, confirm the region, and press **Create bucket**.

Nothing in either bucket is ever public. CloudFront reaches in as a service principal named by the bucket policy, which is why Block Public Access stays fully on and why the buckets are unreachable until 11.3.

Creating `examprep-staging-media` is on its own enough to turn the API's storage probe green:

```
curl -fsS https://api.staging.examprep.iace.co.in/health
```

`"storage":{"status":"up"}`, and the document's own `status` goes from `degraded` to `ok`. Nothing else in this section is needed for that — the probe is a `HeadBucket` call, and the bucket now exists.

11.2 Three distributions

Three distributions, created one at a time from the same form. **Build the student one first and in full** — the other two are the same form with three fields changed, so once you have done it once the remaining two take two minutes each.

Create the origin access control first, once, before any distribution: CloudFront → **Origin access** → **Create control setting** → name it `examprep-s3`, leave **Sign requests (recommended)**, choose **S3** as the origin type, **Create**. Nothing in an OAC names a bucket or a distribution — it is purely "sign requests to S3 with SigV4" — so one covers both buckets and all three distributions, and each distribution below becomes a dropdown selection instead of another modal. You can equally create one inline from the distribution form, or attach one afterwards from **Origins** → **Edit**; doing it up front is just fewer steps. **Sign requests** is also the only setting under which CloudFront talks to S3 over HTTPS unconditionally.

CloudFront → **Distributions** → **Create distribution**. The form has four parts, preceded by one decision the console puts in front of you.

Choose a plan — Free on staging, pay-as-you-go on production

The console offers flat-rate pricing plans and opens on a **Free** tier at $0 a month. Take it **for the three staging distributions**, and take pay-as-you-go for production's three.

A "Free plan" is not the free option — pay-as-you-go is, and it is roughly three times larger. That is the thing to get straight before the names mislead you: the free tier is 1 TB and 10 M requests across the account, where three Free plans come to 300 GB and 3 M. Production therefore stays on pay-as-you-go precisely *because* it is the environment that needs free capacity most.

The plans go on staging for one reason: the free tier is counted per ACCOUNT and applies only to pay-as-you-go usage, so every byte staging serves, and every request a load test makes, would otherwise come out of the same 1 TB production draws on. Three Free plans is exactly the account quota, and spending it here walls that traffic off. Production then inherits the whole free tier uncontested — more than it would get from the plans themselves.

	Free flat-rate plan	Pay-as-you-go
Requests per month	1 M, **per distribution**	10 M, account-wide, always free
Data transfer per month	100 GB, **per distribution**	1 TB, account-wide, always free
CloudFront Functions	included	2 M invocations free
How many you may have	**3 per AWS account**, a quota that cannot be raised	unlimited
Past the allowance	no charge, but sustained excess degrades delivery to fewer and more distant edge locations	standard rates

Three Free plans come to 300 GB and 3 M requests against the free tier's shared 1 TB and 10 M, so as a pure allowance the plans are the smaller offer. They are worth taking anyway because an allowance staging cannot reach is worth more than a larger one it shares with production.

It uses the whole account quota. One plan covers one distribution, the cap is three, and it cannot be raised. Production's three distributions therefore stay on pay-as-you-go by necessity as well as by choice.

A plan makes an AWS WAF web ACL mandatory and permanent. The documentation is explicit that a distribution on a pricing plan must have one attached and that it cannot be removed or disassociated without switching back to pay-as-you-go. So Part 3 below splits: **enable security protections on the staging distributions** — the plan includes the web ACL and five rules at no charge — and decline it on production's.

Custom cache policies, origin request policies and response header policies need Business tier or above. Nothing here uses them — `CachingOptimized` is an AWS *managed* policy and is available on every tier — but on staging it is a wall with no warning in front of it, and the two environments stop being configured identically the day you need one.

A load test can reach 1 M requests. §20 drives the asset tier at event scale, and a plan's allowance is per distribution per month. There is no overage charge, and the documented behaviour is that a first spike up to three times the allowance is absorbed; sustained excess degrades edge routing rather than billing you. On staging that is an acceptable outcome, which is the other reason the plans belong here rather than on production.

Exceeding is recoverable, but not symmetrically. Upgrading a tier is always available, takes effect immediately, is prorated, and restores full delivery; downgrading needs usage already back inside the lower allowance and waits for the next billing cycle. Cancelling is the cheaper escape on staging — a Free plan

cancels immediately and the distribution falls straight back on the account-wide free tier. What does ratchet is eligibility: AWS states that recent usage above a tier's allowance can disqualify you from signing up for or downgrading to that tier. **Run §20's load test before the Free plans go on**, or expect the distribution it hits to end up on Pro.

What a Free plan would actually add is 5 GB of S3 storage credits. The real bill for all three distributions on pay-as-you-go is zero for a long time: a student loads the shell once, the hashed assets cache for a year, and even an 8,000-candidate event moves a few tens of gigabytes.

None of this is a one-way door, and one plan is worth revisiting later. A distribution can join a plan by being updated, and cancelling returns it to pay-as-you-go — immediately on Free, at the end of the billing cycle on a paid tier. The tier worth remembering is **Pro, $15 a month per distribution for 50 TB and 10 M requests**: past roughly 1.14 TB in a month it beats pay-as-you-go overage at $0.109/GB, so if question images turn out to be heavy, a Pro plan on the media distribution alone is the answer.

A plan covers one distribution, and each one chooses independently. Only the Free tier is capped — three per account; paid plans run to a quota of 100, so you upgrade the single distribution that is constrained rather than the environment. Upgrades are immediate and prorated; a downgrade waits for the next billing cycle and needs usage already inside the lower allowance, so one heavy month is not instantly reversible. Three things to know before you do it: a distribution on a plan cannot be deleted until the plan is cancelled, disabling a distribution does not stop its plan charges, and the web ACL a plan attaches cannot be removed while the plan is attached.

Part 1 — Origin

Field	Value	Why
Distribution name	`examprep-staging-student`	A label for you; nothing depends on it
Origin domain	`examprep-staging-spas.s3.ap-south-1.amazonaws.com`	Pick it from the dropdown, never type it
Origin path	`/student`	Leading slash, no trailing slash — see below
Name	leave as filled	Derived from the origin domain
Origin access	**Origin access control settings (recommended)**	The private-bucket mechanism
Origin access control	pick `examprep-s3` from the dropdown	Leaves a job behind, see 11.3
Enable Origin Shield	**No**	An extra caching layer and an extra bill, for one small origin

Pick the origin from the dropdown, and take the bucket, not the website endpoint. Click the field and your buckets are listed. If the console then shows a notice offering to use the S3 **static website endpoint** instead, decline it. That endpoint is plain HTTP, supports no origin access control, and using it means turning Block Public Access off — the opposite of this design. The right value ends in `.s3.ap-south-1.amazonaws.com`; the wrong one ends in `.s3-website-ap-south-1.amazonaws.com`.

Origin path is the whole reason one bucket can serve two websites. The publish script uploads the student site to `s3://examprep-staging-spas/student/…` and the admin site to `…/admin/…`. CloudFront prepends the origin path to every request it forwards, so a browser asking this distribution for `/index.html` makes CloudFront ask S3 for `/student/index.html`. Get it wrong and every request 404s at the origin while CloudFront itself looks healthy.

Origin access control replaces a public bucket. CloudFront signs each request to S3 as the `cloudfront.amazonaws.com` service principal, and the bucket policy allows exactly that principal from exactly this distribution. Nobody can reach the bucket by its own URL, with or without guessing a key. When you click

Create new OAC the defaults are already right — signing behaviour *Sign requests (recommended)*, origin type S3 — so name it and create it.

Part 2 — Default cache behavior

Field	Value	Why
Path pattern	`Default (*)`	Fixed, not editable
Compress objects automatically	**Yes**	JS and CSS are text; this is most of the transfer saving
Viewer protocol policy	**Redirect HTTP to HTTPS**	A student typing the host without a scheme still arrives on HTTPS
Allowed HTTP methods	**GET, HEAD**	A website is read-only; the API is a different host entirely
Restrict viewer access	**No**	Signed URLs are for private files, and these are the websites
Cache policy	**CachingOptimized**	AWS managed; see below
Origin request policy	**None**	Nothing about the viewer needs to reach S3
Response headers policy	**None**	Images in `<img>` need no CORS header; the API sets its own
Function associations	leave empty for now	The function does not exist yet — 11.4 comes back here

`CachingOptimized` **is the one to take and it is safe here**, even though it caches aggressively, because the publish script makes it safe: asset filenames carry a content hash and are uploaded `max-age=31536000,immutable`, while `index.html` is uploaded `no-cache`. So the big files cache for a year and the one file that must never go stale is re-fetched every time. The policy also forwards no cookies, no query strings and no headers, which is what keeps the hit rate high.

Leave Function associations alone on this pass. The routing function is created in 11.4, and you cannot associate a function that does not exist yet. You will come back to this same screen through **Behaviors → Default → Edit**.

Part 3 — Web Application Firewall

This is the one field that differs between the two environments, and it follows from the plan decision above rather than from any security judgement.

On staging, enable security protections. The three distributions are on Free plans, and a distribution on any pricing plan *must* carry a web ACL — it cannot be removed while the plan is attached. The plan covers it, so it costs nothing: a web ACL and up to five rules at $0. Accept the console's default rules; nothing here needs tuning.

On production, do not enable them. Those distributions are pay-as-you-go, where the same web ACL is roughly $5 a month plus per-request charges. They are two static websites behind a CDN with no forms and no writes, so there is nothing for a firewall to protect, and the API — which does take writes — is on Box A behind Caddy and never passes through CloudFront at all.

One upside of the split worth noting: blocked requests never count against a plan's usage allowance, so on staging the web ACL's rate limiting also protects the allowance itself.

Part 4 — Settings

Field	Value	Why
Price class	**Use North America, Europe, Asia, Middle East, and Africa**	Production only — absent on a plan, see below

Field	Value	Why
Alternate domain name (CNAME)	leave empty	A CloudFront domain is enough for staging; §19 adds one for production
Custom SSL certificate	leave empty	Follows from no CNAME; the default `*.cloudfront.net` certificate is used
Supported HTTP versions	**HTTP/2** (default)	HTTP/3 is fine too and costs nothing either way
Default root object	`index.html`	Without it, `/` returns a 403 from S3 and the site looks broken
Standard logging	**Off**	Access logs to S3 cost storage and nobody reads them on staging
IPv6	**On** (default)	Free

Price class does not appear on a staging distribution, and that is correct. A flat-rate plan includes delivery from all 750+ edge locations at every tier, so there is no per-region rate left to control and the console drops the field. Those distributions sit at `PriceClass_All` and it costs nothing.

On production, where the distributions are pay-as-you-go, it is a real bill and the default is the expensive one. *Use all edge locations* is every point of presence on earth. Your students are in India, and the option that includes Mumbai is the **three-region** one — North America, Europe, Asia, Middle East, and Africa. Do not pick *North America and Europe only*: it is the cheapest row, and it excludes India, so every request would cross an ocean. Changeable later with no downtime.

Default root object is `index.html` and it is only about the bare `/`. A request for `/` is not a request for a file, so without this setting CloudFront asks S3 for the directory and gets a

1. It does not help with `/tests/abc` — that is what the function in 11.4 is for.

Press **Create distribution**. It takes a few minutes to leave *Deploying* and reach **Enabled**; you do not have to wait before creating the next one.

The other two

The admin distribution is the same form with one field different:

Field	Value
Distribution name	`examprep-staging-admin`
Origin path	`/admin`
Origin access control	**reuse the OAC you just created** — do not create a second

The media distribution serves question images rather than a website, so three fields change:

Field	Value	Why
Distribution name	`examprep-staging-media`	
Origin domain	`examprep-staging-media.s3.ap-south-1.amazonaws.com`	The other bucket
Origin path	**empty**	Media keys are already full paths — `questions/images/abc.png`

Field	Value	Why
Default root object	**empty**	There is no `index.html`; nobody browses this host
Origin access control	reuse the same OAC	

One OAC is enough for all three. It carries no bucket name and no distribution name — it is just "sign requests to S3 as CloudFront" — so the same one works everywhere, and the thing that actually grants access is the bucket policy in 11.3.

Write down six values

Each distribution gives you two identifiers and you need both kinds:

	Looks like	Used for
Distribution **ID**	`E1A2B3C4D5E6F7`	`publish-spas.sh`, cache invalidations (11.6)
Distribution **domain name**	`d111111abcdef8.cloudfront.net`	`MEDIA_BASE_URL` and `CORS_ORIGINS` (11.5)

They are not interchangeable, and the commonest mistake in 11.6 is passing a domain where the script wants an ID. Both are on the Distributions list page, in the **ID** and **Domain name** columns.

11.3 The bucket policies

Each bucket needs a policy saying which distributions may read it. S3 → the bucket → **Permissions** → **Bucket policy** → **Edit** → paste → **Save changes**.

For `examprep-staging-media` there is one distribution, so the console's **Copy policy** output works unchanged.

For `examprep-staging-spas` it does not, and this is the trap. Two distributions read that bucket, and the console generates a policy for one distribution at a time. Paste the student policy, then paste the admin policy, and the second *replaces* the first — a bucket policy is one document, not a list you append to. The student site then returns 403 on every file, with nothing on the CloudFront side to suggest why.

Write one policy naming both. `AWS:SourceArn` accepts a list, and `StringEquals` against a list means *any of these*:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowCloudFrontServicePrincipal",
      "Effect": "Allow",
      "Principal": { "Service": "cloudfront.amazonaws.com" },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::examprep-staging-spas/*",
      "Condition": {
        "StringEquals": {
          "AWS:SourceArn": [
            "arn:aws:cloudfront::598927569098:distribution/STUDENT_DISTRIBUTION_ID",
            "arn:aws:cloudfront::598927569098:distribution/ADMIN_DISTRIBUTION_ID"
          ]
        }
      }
    }
  ]
}
```

`598927569098` is the account id; substitute the two distribution ids. The `/*` on `Resource` is required — a policy on the bucket ARN alone grants nothing on the objects inside it.

11.4 The single-page-app routing rule

A SPA has routes like `/tests/abc/attempt` that are not files. Without help CloudFront asks S3, gets a 404, and the student sees an error page.

Do not use the "custom error response 404 → /index.html → 200" trick. It works for page routes and also turns genuinely missing files into HTML — so a chunk that failed to upload is served as HTML and the browser reports a syntax error in a file that looks fine.

CloudFront → **Functions** → **Create function**. Name `spa-routing`, runtime **cloudfront-js-2.0**. On the **Build** tab replace the body with:

```
function handler(event) {
  var request = event.request;
  var uri = request.uri;
  // A dot in the last segment means a real file: let S3 answer, 404 included.
  if (uri.indexOf('.') === -1) request.uri = '/index.html';
  return request;
}
```

Save changes, then the **Publish** tab → **Publish function**. A function that is saved but not published is not running anywhere.

Then associate it, from the Publish tab's *Associated distributions* → **Add association**, or from each distribution's **Behaviors** → Default → Edit → Function associations:

Distribution	Event type	Function
Student	Viewer request	`spa-routing`
Admin	Viewer request	`spa-routing`
Media	*none*	

The media distribution must not have it. Media keys are paths like `questions/images/abc123.png`; rewriting anything without a dot to `/index.html` on a bucket that has no `index.html` would turn every miss into a 404 from the wrong place.

The rewrite happens *before* the origin path is prepended, so a student request for `/tests/abc/attempt` becomes `/index.html` and CloudFront asks S3 for `/student/index.html`.

11.5 Write the domains into the API

Both values are CloudFront domains, and neither is known until 11.2 finished.

```
MEDIA_BASE_URL=https://dmedia111111.cloudfront.net
CORS_ORIGINS=https://d111111abcdef8.cloudfront.net,https://d222222abcdef8.cloudfront.net
```

Edit them in `~/.examprep/staging.env` on your laptop and in `deploy/.env` on Box A — the two are separate files and the one on the box is what runs. Then, from `~/iace/deploy`:

```
sudo docker compose up -d
```

`up -d` recreates the containers whose environment changed and leaves the rest alone. No rebuild: nothing in the image changed.

No bucket name in `MEDIA_BASE_URL` — §7.4 explains why, and it is the one that 404s every image. The API builds a URL as `${MEDIA_BASE_URL}/${key}`, and the distribution's origin is already the bucket, so a bucket name in the base produces `…/examprep-staging-media/questions/…`, a key that does not exist.

`CORS_ORIGINS` **is the other biter**: exact origins, scheme included, no trailing slash, comma separated with no spaces. Wrong, and the site loads perfectly while every request fails, visible only in the browser console. Add `http://localhost:5173` if you want to run a SPA locally against this box.

11.6 Publish

From your laptop — it builds the sites, so it needs the repo and Node, not the box:

```
./deploy/publish-spas.sh https://api.staging.examprep.iace.co.in examprep-staging-spas
E111111ABCDEF E222222ABCDEF
```

The two `E…` are **distribution IDs**, not domains, and student comes before admin. Three things in that script are not obvious:

- `VITE_API_URL` **is baked in at compile time.** The API address is not a runtime setting for a website; it is compiled into the JavaScript. An environment is therefore a *build*, and a staging bundle can never be promoted to production.
- **Hashed assets upload first, the shell second.** The other order serves, for a few seconds, an `index.html` referencing chunks that are not there — and every visitor in that window gets a blank page.
- **Old asset files are never deleted.** A tab opened before the deploy still lazy-loads chunks under their old hashed names. Keeping them costs about a cent a year.

11.7 Check it

```
curl -I https://d111111abcdef8.cloudfront.net/
curl -I https://d111111abcdef8.cloudfront.net/tests/does-not-exist
curl -I https://d111111abcdef8.cloudfront.net/assets/nope.js
```

The first two are `200` with `content-type: text/html` — the second proves the function is associated and published. The third is `404`, which proves it is the *right* function and not the error-response trick: a missing asset must stay missing.

A `403` on the first means the bucket policy, not the function — and on the student site specifically, it usually means the admin policy overwrote it. Re-read 11.3.

---

12. Verify, end to end

```
curl -s https://api.staging.examprep.iace.co.in/health/ready | jq
curl -sI https://api.staging.examprep.iace.co.in/health | head -1

# Caddy routes to the right container. 401 from both is correct: unauthenticated, but it arrived.
curl -s -o /dev/null -w '%{http_code}\n' https://api.staging.examprep.iace.co.in/me/leaderboard
curl -s -o /dev/null -w '%{http_code}\n' https://api.staging.examprep.iace.co.in/auth/me

# Metrics answer with the token and refuse without it.
curl -s -o /dev/null -w '%{http_code}\n' https://api.staging.examprep.iace.co.in/metrics
curl -s -H "authorization: Bearer $METRICS_TOKEN" https://api.staging.examprep.iace.co.in/metrics |
head -5
```

The real test is the product: sign in to the admin site, read `devCode` out of the network tab, create one student, create one test, publish it; then sign in to the student site as that student and start the test. Fifteen minutes, and it exercises everything.

Tell the tester three things: the student URL, the admin URL, and that **the OTP is in the network response, not in any SMS** — the field is `devCode` on the OTP request call. Without that last sentence they will sit waiting for a text message.

Snapshot the box. EC2 → the instance → Actions → Image and templates → Create image. A replacement from that image needs only the env file from SSM, which is a three-minute recovery instead of repeating this document.

12.6 Custom domains for the sites — later, 40 minutes

1. **ACM → us-east-1** → request a certificate for `staging.examprep.iace.co.in` and `admin.staging.examprep.iace.co.in`. It must be us-east-1; CloudFront reads certificates from nowhere else, whatever region your buckets are in. **Choose Disable export.** The form offers an exportable certificate and it is the only paid choice on the page: $7 per domain name at issuance and again at every renewal, which on a three-name certificate renewing about every 198 days is roughly $42 a year. Export hands you the private key, for terminating TLS somewhere AWS does not manage — your own nginx, an on-prem box, another cloud. CloudFront reads the certificate natively and the key never leaves AWS. The setting is fixed at issuance and cannot be changed later, which costs you nothing: an exportable certificate, if ever needed, is a separate request.
2. Add the validation CNAMEs at GoDaddy, with `.iace.co.in` stripped from the Name field. These are proof of ownership, not routing — ACM's certificate authority queries DNS for a name only it could have told you, and finding it proves whoever asked for the certificate can edit the zone. **Leave them in place forever.** ACM re-validates against them at every automatic renewal, now roughly every 198 days; tidy them away after issuance and the renewal fails silently, the certificate expires, and every site starts throwing TLS errors on a day nobody touched anything. They are a different record from step 5's — a token-prefixed subdomain pointing into `acm-validations.aws`, where step 5 points the real hostname at CloudFront.
3. Wait for **Issued**.
4. On each distribution: Alternate domain name → the hostname; Custom SSL certificate → the ACM one. Wait for Deployed.
5. Add the two CNAMEs from §8.3.
6. Add the new origins to `CORS_ORIGINS` and restart.

No rebuild is needed, and that is worth understanding. `VITE_API_URL` is the only compile-time variable either app has, and it holds the **API's** address, not the site's own. Moving a site to a new hostname changes nothing

inside its bundle. What must change is `CORS_ORIGINS` — miss it and the site loads perfectly while every request fails.

---

13. Deploying after today — the pipeline

Seven workflows. The branch is the environment.

```
feat/<thing>      short-lived, optional while there is one developer
   ↓
staging           every session commits here; deploys staging on push
   ↓  a reviewed promotion, when production exists
main              production
```

Workflow	Trigger	What it does
`ci.yml`	push to either branch, and PRs	verify, database, docs-check
`build-api.yml`	push touching `apps/api`, `packages`, `prisma`	Builds both targets natively on arm64, pushes to ECR as `<sha>` plus `staging` or `prod` by branch. **Does not deploy**
`staging.yml`	push to `staging`	Deploys at once. Works out from the diff whether the push needed the API, the SPAs or both
`production.yml`	cron 02:00 IST, or Run workflow	API then SPAs, behind the production environment's reviewer. **Dormant until §19**
`_deploy-api.yml`	reusable	The API body; both environments call it, so they cannot drift
`_publish-spas.yml`	reusable	The SPA body, same reason
`scale.yml`	Run workflow	§16, without a shell

The front end auto-deploys; the API waits for the night. A SPA release is a file copy. An API release runs migrations and recreates containers with a gap (§13.3), so it is a small outage at a moment nobody chose.

`packages/contracts` **is deliberately absent from the front end's trigger.** It is the wire format both sides share, so a change there is an API release and the SPAs follow it through `workflow_run` — never before it, and not at all if it failed. The rule behind that is **the API goes before the SPAs**: an old tab keeps its old bundle until every tab closes, so the API must serve the previous client anyway; the reverse is not survivable. A new bundle meeting an old API can send a submit field the old API strips, and the answers go with it.

No AWS key exists in GitHub. Each run exchanges its OIDC token for a session on `examprep-github-actions`, whose trust names exactly four subjects — both branch refs and both environments. On a **public** repository that scoping is the whole difference: `repo:…:*` would let a workflow on any fork assume the role.

Rolling back is a tag, not a rebuild: Run workflow with the previous commit's sha.

The manual path still works and is the fallback when CI is down: `git pull && docker compose up -d --build` on the box. `deploy/compose.yml` keeps both `image:` and `build:` for that reason.

13.1 What it costs to keep, if the repo goes private

`ubuntu-24.04-arm` runners are free on **public** repositories only. A private repo on the Free plan also loses branch protection and environment reviewers, and caps at 2,000 Actions minutes — and a QEMU-emulated build at 45–90 minutes would eat most of that.

	Monthly
CodeBuild as an ephemeral ARM runner, ~20 builds	**~$0.60**
A self-hosted runner on an always-on `t4g.small`	$8.18

	Monthly
GitHub Team	$28

CodeBuild registers a container only while a job runs: `arm1.small` at **$0.002 a minute**, native ARM, and **no GitHub minutes at all**. It needs a classic token with `repo` and `admin:repo_hook`, `aws codebuild import-source-credentials`, a project on `ARM_CONTAINER` / `aws/codebuild/amazonlinux-aarch64-standard:4.0` with `privilegedMode=true`, a webhook filtered to `WORKFLOW_JOB_QUEUED`, and one line:

```
runs-on: codebuild-examprep-builder-${{ github.run_id }}-${{ github.run_attempt }}
```

That label must match the project name and carry both parts — wrong, and the job queues for ever with no error, which is the worst failure shape there is. Nothing else in the workflow changes. Only `ci.yml` stays on GitHub's runners, at ~140 of the 2,000 minutes.

Self-hosted runners are only safe once the repo is private, because a fork's pull request can otherwise run code on your builder.

Branch protection and the reviewer gate have no AWS substitute. The workarounds: a local `pre-push` hook refusing a non-fast-forward to `main`, and making `production.yml` dispatch-only, so the manual trigger *is* the gate.

---

14. Caddy — what it is, and every line of the Caddyfile

Caddy is the only thing on Box A that listens to the internet. Three jobs: it gets and renews the certificate, it terminates TLS, and it forwards each request to whichever container should answer.

```
{
        email {$ACME_EMAIL}
}
```

The global block. One setting: where Let's Encrypt writes if a renewal is failing. `{$VAR}` is Caddy's environment-variable syntax and compose passes it in.

```
{$API_HOST} {
```

A **site block**, and the hostname *is* the configuration — Caddy sees a real name and automatically obtains a certificate, redirects 80 to 443, and serves HTTP/2 and HTTP/3. There is no "enable TLS" line because there is no way to express "do not".

```
        @exam path /me/tests/*/attempt /me/tests/*/brief /me/tests/*/paper \
                   /me/attempts/*/paper /me/attempts/*/submit /me/attempts/*/state /me/leaderboard
```

A **named matcher** listing the exact leaf paths the exam role serves.

Why exact paths and not a prefix. The split is not prefix-clean. `GET /me/performance` is exam, but `/me/performance/report`, `/me/performance/days` and `/me/performance/series` are core. `/me/attempts/*/question-report` is core, under the very prefix three exam routes use. A wildcard would route live traffic to the wrong container — silently, as a 404 rather than an error.

```
        reverse_proxy @exam {
                dynamic a { name exam; port 3000; refresh 10s }
                fail_duration 30s
                max_fails 3
        }
```

`dynamic a` **is what makes scaling work**: it re-resolves the DNS name `exam` every 10 seconds, and Docker's resolver returns *every* container of that service. `--scale exam=4` is load-balanced across four within ten seconds, with no edit here and no restart.

`fail_duration 30s` with `max_fails 3` is a **passive** health check: a container that fails three requests leaves rotation for thirty seconds. Passive, not active — Caddy does not poll `/health`; Docker's own healthcheck does.

```
        reverse_proxy { dynamic a { name core … } }
```

No matcher, so this is the catch-all. Order matters: the first matching `reverse_proxy` wins, so the specific one must come first.

```
        log { output stdout; format json }
```

One JSON object per request to stdout. Docker captures it, Alloy ships it, and the same 10 MB × 3 rotation caps it. It used to be a file in a volume — that was before anything existed to carry it off the box.

14.1 Changing a route

Add a route to the exam role and its path must be added here, or it falls to `core`, which does not register it, and the student gets a 404 that looks like a missing feature.

```
sudo docker compose ... exec caddy caddy validate --config /etc/caddy/Caddyfile
sudo docker compose ... restart caddy
```

Validate first. A Caddyfile with a syntax error stops the container, and then nothing is serving.

14.2 What it gives up, and the certificate volume

One program on one box. A load balancer is multi-node and self-healing; this is not, so **a reboot is an outage**. At ten to thirty events a month that is scheduled around rather than solved. The one escape hatch kept open is a warm standby behind a Route 53 health check — a $0.50 health check and a second Caddy, not a load balancer and not a scheduler.

`caddy-data` holds the issued certificates. **Do not delete it casually**: Let's Encrypt rate-limits five duplicate certificates per hostname per week, and a few careless `down -v` runs lock you out of your own hostname for a week.

---

15. Grafana Cloud — logs and metrics

The `alloy` container already in `deploy/compose.yml` does both: every container's stdout into Loki, and the API's `/metrics` into Prometheus. Nothing in the API knows it exists; it reads what Docker already captured.

1. Sign up at grafana.com — the free tier is 50 GB of logs and 10,000 series, far more than this produces.
2. **The credentials are not in the Grafana you just logged into.** Signup drops you at `<name>.grafana.net`, which is the Grafana instance — dashboards and Explore. The connection details live in the account portal at **grafana.com**, a separate site, same login. Sign in there and you land on an **Overview** page with a tile per signal.
3. **Loki tile → Details**, section *Sending Logs to Grafana Cloud using Grafana Alloy*: the URL and a numeric username.
4. **Prometheus tile → Details**: the **Remote Write Endpoint** and a *different* numeric username.
5. **Security → Access Policies → Create access policy**, with **both** `logs:write` and `metrics:write` ticked, then **Add token**. One token serves both signals, so it goes in `LOKI_PASSWORD` and `PROM_PASSWORD` alike. Anything labelled *API Keys* is the legacy mechanism access policies replaced.
6. **Both URLs are full push paths**, because `config.alloy` hands them to `loki.write` and `prometheus.remote_write` unchanged — nothing appends a path:

`` `LOKI_URL=https://logs-prod-<region>.grafana.net/loki/api/v1/push PROM_URL=https://prometheus-prod-<region>.grafana.net/api/prom/push` ``

A bare hostname returns a 404 that reads exactly like an auth failure.

> `PROM_USERNAME` is not `LOKI_USERNAME`. Grafana Cloud numbers each signal separately, and pasting one into the other fails with a 401 that reads exactly like a bad token.

```
sudo docker compose -f deploy/compose.yml --env-file deploy/.env up -d alloy
sudo docker compose -f deploy/compose.yml --env-file deploy/.env logs --tail 30 alloy
```

In Grafana → Explore → Loki:

```
{env="staging"}
{env="staging", service_name="exam"}
{env="staging", level="error"}
{service_name="caddy"} | json | status >= 500
```

There are exactly three labels — `env`, `service_name`, `level` — on purpose. A label is an index in Loki, and a high-cardinality one makes queries slow and the bill large. Everything per-request stays in the line and comes out at query time:

```
{service_name="exam", level="error"} |= "bug=" | pattern "<_>bug=<bug> <_>" | sum by (bug)
```

In Explore → Prometheus:

```
up{role="exam"}
rate(iace_http_request_duration_seconds_count{status=~"5.."}[5m])
histogram_quantile(0.95, sum by (le) (rate(iace_http_request_duration_seconds_bucket[5m])))
```

`up == 0` is a container Alloy cannot scrape, and `count(up{role="exam"})` after `--scale exam=4` should read 4 within fifteen seconds — nothing is listed anywhere, it is all discovered off the Docker API, the same way Caddy's upstreams are.

The metric prefix is `iace_`, not `examprep_`, and deliberately: it comes from `PREFIX` in `apps/api/src/common/metrics/metrics.service.ts`, and renaming it orphans every dashboard and alert built on it.

Alloy is root on this box. It reads the Docker API to learn which containers exist and what compose calls them, and `:ro` on a socket mount protects the file rather than the API behind it — anything that can reach that socket can start a privileged container. That is the price of `service_name` being a name rather than a 64-character id.

15.1 Four alerts, not forty

Alert	Query	Threshold	Pending	No data
API down	`count(up{env="staging", role="exam"})`	below 1	2m	**Alerting**
5xx rate	`sum(rate(iace_http_request_duration_seconds_count{env="staging", status=~"5.."}[5m])) / sum(rate(iace_http_request_duration_seconds_count{env="staging"}[5m]))`	above 0.01	5m	OK
Queue backing up	`max(iace_queue_oldest_waiting_seconds{env="staging"})`	above 300	5m	OK
Valkey memory	`max(iace_redis_memory_ratio{env="staging"})`	above 0.8	5m	OK

They watch, in order: the thing that ends an exam; something breaking under load; scoring not draining, which students see as missing results; and a full Valkey, which under `noeviction` makes writes **fail** rather than slow down.

There is no `iace_http_requests_total` — the error rate comes off the latency histogram's `_count` series, which carries the `status` label and is what its help text describes.

`API down` **must set No data to Alerting**, and it is the only one that should. `up` is produced by the scraper, so a container that vanishes takes its target with it and the query returns no series rather than zero — a threshold alone stays silent through exactly the outage the rule exists to catch. The other three go the other way: at 3am with no traffic the 5xx ratio is 0/0, and paging for an idle system is how an alert set earns its way into being ignored.

A contact point must exist first or a firing rule notifies nobody. Grafana Cloud pre-creates `grafana-default-email` at your signup address — confirm it under Alerting → Contact points.

Plus the two free **AWS Budgets** — one at your monthly total, one anomaly detector. Every alert above watches something that is running; none watches something that should have **stopped**, and on a first AWS account a forgotten instance costs more than the mistakes the alerts catch. **Neither is yours to create**: `budgets:ModifyBudget` and `ce:CreateAnomalyMonitor` are both an explicit deny on the developer user, so ask the account owner. The CloudWatch alarm below is allowed, and so is everything in §19.

Also worth an alarm: `CPUSurplusCreditsCharged`. `t4g` is burstable and unlimited mode is the default, so past the 20%-per-vCPU baseline you are billed rather than throttled — about $0.04 per surplus vCPU-hour. The right trade for an exam, and the single largest gap between the expected and the worst-case invoice.

---

16. Scaling

Four dials, in the order to reach for them.

More containers — seconds, free:

```
sudo docker compose -f deploy/compose.yml --env-file deploy/.env up -d --scale exam=4
```

Caddy picks them up within ten seconds and Alloy starts scraping them. **Nothing is configured anywhere.** `scale.yml` does the same from Actions, with guards.

The worker is excluded, always. `--scale worker=2` puts two sweeps over the same Valkey keys with nothing coordinating them. If scoring ever needs to scale, the answer is a fourth role registering only the scoring processor — not a second worker.

Watch the database when you do. Three containers at a pool of 25 is 75 of the instance's ~112, so the **third** API container of any role is the trigger for `db.t4g.small` — not the second exam one. `scale.yml` computes this against the live instance class and refuses past 90% of the ceiling.

A bigger box — two minutes, hourly billing: stop, change instance type, start. The Elastic IP and EBS survive.

Size	vCPU	RAM	When
`t4g.small`	2	2	Staging once images come from ECR
`t4g.medium`	2	4	Staging today — you build on the box
`t4g.large`	2	8	Production baseline
`t4g.xlarge`	4	16	A scheduled event
`t4g.2xlarge`	8	32	~8× the 10K ceiling

Pre-warming is deliberate, not automatic. There is no autoscaling group. Resizing before a window goes on the release calendar — the alternative is discovering the need while 8,000 people are typing.

Box B and RDS: `t4g.micro` → `small` before the first full-scale load test, → `medium` when production carries 8,000 live sittings. What sizes Box B is not the dataset but the **AOF rewrite fork**: Valkey rewrites its log by forking, and copy-on-write can push resident memory towards double the dataset. CPU never decides — Valkey is single-threaded and an event is ~960 ops/s against a core that does 100,000.

17. PostgreSQL — backups and restore

Point-in-time restore, 7 days, turned on in §5. RDS continuously ships write-ahead logs, so you can restore to any second in that window. Free up to your allocated storage, and it is what saves you from a bad migration or a wrong `DELETE`.

A restore does **not** overwrite the instance — it creates a *new* one from the chosen moment, which you then point the application at. Slower to think about and much safer.

```
aws rds restore-db-instance-to-point-in-time \
  --source-db-instance-identifier examprep \
  --target-db-instance-identifier examprep-restored \
  --restore-time 2026-10-04T09:30:00Z \
  --db-subnet-group-name examprep-db-private \
  --vpc-security-group-ids sg-0d1e4e694a408945d \
  --no-publicly-accessible
```

A manual snapshot before anything risky — these are kept until you delete them, unlike the automatic ones:

```
aws rds create-db-snapshot --db-instance-identifier examprep \
  --db-snapshot-identifier examprep-pre-release-$(date +%Y%m%d-%H%M)
```

A logical dump, which neither of those replaces. A snapshot and a PITR both restore a whole instance; neither lets you pull one table back, read the data elsewhere, or survive losing the AWS account:

```
pg_dump "$DATABASE_URL" --format=custom --no-owner --no-acl --file=/tmp/examprep_$(date +%F).dump
aws s3 cp /tmp/examprep_$(date +%F).dump s3://examprep-staging-media/backups/ --storage-class
STANDARD_IA
rm /tmp/examprep_*.dump
```

Rehearse it once, on staging, this month. A backup nobody has restored is not a backup. Restore to a new instance, point the box at it, confirm the admin can sign in, delete it. An hour, and it turns the procedure above from a document into something you have done. That is also why staging runs 7-day PITR at all.

Never restore production into staging. Real students' names, numbers and marks do not belong in a lower-security environment — and while the two share one instance that rule works harder than usual.

Single-AZ is an assumption, not a decision. The cost table was built on it and nobody argued it. Single-AZ makes an instance or zone failure a restore — tens of minutes, a new endpoint, `DATABASE_URL` repointed, and mid-event that sitting is gone. Multi-AZ fails over in 60–120 seconds on the same endpoint for the instance price again plus the storage again: `db.t4g.small` goes from $30.66 to ~$61.32. **Decide it when production is sized from the load test** — it is a modify with no data loss, so unlike a bucket name it is not a now-or-never choice.

---

18. Valkey — connecting and looking inside

From Box B — `REDISCLI_AUTH` is already in the container's environment, so the shell is authenticated and the password never reaches a command line or shell history:

```
sudo docker compose -f deploy/valkey/compose.yml exec valkey-staging valkey-cli
```

From your laptop, through Box A, with no inbound rule and no VPN:

```
aws ssm start-session --target <box-a> \
  --document-name AWS-StartPortForwardingSessionToRemoteHost \
  --parameters '{"host":["10.0.0.32"],"portNumber":["6380"],"localPortNumber":["16380"]}'
```

```
INFO memory          # used_memory_human against maxmemory — the number that matters
INFO persistence     # aof_enabled 1, aof_last_write_status ok
DBSIZE
SCAN 0 MATCH bull:* COUNT 20
```

Never run `KEYS *` on a box with live sittings. Valkey is single-threaded and `KEYS` walks the whole keyspace, blocking everything else for as long as it takes. `SCAN` is the one that is safe.

Setting	Value	Why
`maxmemory-policy`	`noeviction`	BullMQ requires it, and this holds live answers. Any eviction policy silently drops a student's work. The API refuses to boot in production against anything else
`maxmemory`	always explicit	Unset, Valkey grows until the kernel kills it, losing up to a second of answers
`appendonly`	`yes`, every second	A restart loses at most that second
`requirepass`	one per process	§6.3

One process per environment, not one process with two database indexes. An index would separate the keyspaces — which matters, because BullMQ has no prefix configured and both environments would otherwise share `bull:scoring:*`, letting a production worker take a staging job. But `maxmemory` is per *process*, and under `noeviction` an instance that fills makes writes **fail**. A separate process separates the memory as well as the keys.

When it dies, you lose **in-flight sittings, not scored ones**: answers since the last 60-second flush, plus sessions and OTPs. The AOF replays on start.

Why not ElastiCache. Serverless has no parameter groups, so `maxmemory-policy` cannot be set at all — which BullMQ's own documentation rules out. A node-based `cache.t4g.small` is $23.94 against this box's $4.09 and comes back **empty** after a node failure, because no current ElastiCache engine writes to disk.

19. Production — what is different

Build it only once staging has run a real load test, because the one number you do not have is what size Box A should be.

	Staging	Production
Box A	`t4g.medium` → `small`	`t4g.large`
Box B	shared, port 6380	same box, port **6379**, second process
RDS	`examprep_staging`	`examprep_production`, same instance, `db.t4g.small`
`NODE_ENV`	`development`	`production`
`OTP_SENDER`	`console`	`sms`
Heaps	384 MB each	exam 1536, core 768, worker 768
Hostname	`api.staging.examprep…`	`api.examprep…`
SSM parameter	`/examprep/staging/env`	`/examprep/production/env`
Secrets	generated	**different** generated values
Grafana Cloud	**one stack, shared**	same URLs and usernames, its own token
`ENVIRONMENT`	`staging`	`production`

Grafana Cloud is shared on purpose. Alloy stamps `ENVIRONMENT` into an `env` label, so both environments ship to the same Loki and the same Prometheus and you separate them at query time — `{env="staging"}` against `{env="production"}`. Two stacks would mean two logins and no way to put the two side by side, which is most of what a dashboard is for. Production therefore reuses `LOKI_URL`, `LOKI_USERNAME`, `PROM_URL` and `PROM_USERNAME` unchanged. Give it **its own access policy and token**, with the same `logs:write` and `metrics:write` scopes, so either environment's credential can be rotated without touching the other.

The one thing that must not be copied is `ENVIRONMENT`. Production's `deploy/.env` carries `production`; inherit `staging` from a copied file and both environments' logs land under one label and interleave, with nothing failing to announce it.

What production refuses to start without, each naming its own variable rather than failing at the first login: `connection_limit` on `DATABASE_URL`; an empty `CORS_ORIGINS`; a missing `METRICS_TOKEN`; `TRUST_PROXY_HOPS` at 0; a Valkey that can evict; any of `JWT_ACCESS_SECRET`, `JWT_REFRESH_SECRET` or `PIN_PEPPER` still carrying the published `dev_only_` prefix; `OTP_SENDER=console`, and `sms` without `SMS_PROVIDER_URL`, `MAIL_USER` and `MAIL_PASSWORD`.

It also **warns**, without refusing, when a container's heap does not match its memory limit.

Two database steps that are easy to forget: `CREATE EXTENSION IF NOT EXISTS pg_trgm` as master in the new database, and `REVOKE CONNECT … FROM PUBLIC` on **both** databases — that second one is what stops staging's role opening a connection to production's.

And the hostname is a cutover, not a record. `examprep.iace.co.in` currently points at ThinkExam with students on it (§3.3).

A new service worker installs but does not activate until every tab of the old one closes. There is no `skipWaiting`, deliberately: a bundle swapped under a sitting in progress is worse than a stale tab. Deploying and students seeing the new build are different moments.

A release that bumps `STEM_HASH_VERSION` rehashes the bank in the worker as it boots, logging `Rehashed N question stems`. Until that line an import can miss a duplicate — hold imports until it.

Never during an event window, and never a migration that moves data without rehearsing it against a copy at the previous revision.

19.1 What it costs

Production and staging together, at the sizes §19 names. Every rate below is measured from an actual `ap-south-1` invoice rather than taken from the pricing page.

	What	Monthly
Box A — production API	`t4g.large` + EBS 20 GB + Elastic IP	$38.17
Box A′ — staging API	`t4g.small`, once builds move to ECR	$14.57
Box B — Valkey	one `t4g.medium`, a process per environment	$21.82
Database	RDS `db.t4g.small`, two databases on one instance	$33.28
Frontend, media, DNS, alarms		$3.40
Total		**$110.32**

Add 18% GST: `ap-south-1` bills through AWS India, so about **$130**. It is input credit against the institute's GSTIN, but only if the GSTIN is on the account.

The component rates, for sizing anything not in that table:

	Rate
`t4g.micro` / `small` / `medium` / `large`	$0.0056 / $0.0112 / $0.0224 / $0.0448 per hour
`db.t4g.micro` / `db.t4g.small`	$0.021 / $0.042 per hour
EBS gp3	$0.0912 per GB-month
RDS gp3 storage	$0.131 per GB-month
Public IPv4	$0.005 per hour, **attached or idle**

`t4g.small` **is free until 31 December 2026** — AWS's trial covers 750 hours a month, which is more than one instance running continuously. Staging's API box therefore costs nothing to run today, and about $8.18 a month from January. Budget for the step.

The bill does not move with concurrency. These are fixed instances, so three hundred students and three thousand cost the same. What 3K buys is headroom, not a larger invoice.

Two things are outside the table and both scale with students. CloudFront image bytes are the line §15 calls the one that can move the invoice by an order of magnitude — unmeasured, free to 1 TB, and answered by a $15 Pro plan rather than a re-encode if a real paper turns out heavy. And **SMS**: a student signs in with a code every time, so OTP volume now tracks logins rather than signups.

19.2 Stopping staging between sessions

Staging can be stopped; production cannot. Nobody is using staging between working sessions, so most of its cost is switchable — but less of it than you would expect:

	Stops?	Monthly
EBS, 30 GB across both boxes	**no**	$2.74
RDS storage, 20 GB	**no**	$2.62

	Stops?	Monthly
Elastic IP	**no** — an idle address bills the same	$3.65
S3, ECR, backups	—	$0.25
EC2 and RDS compute	yes	$0
Floor		**$9.26**

Three traps in that. **An Elastic IP costs the same stopped**, and at 39% of the floor it is the largest single line — it exists so the box keeps its address across restarts, and releasing it means re-pointing DNS every time. **A stopped RDS restarts itself after seven days**, whether you want it or not, and bills until you stop it again; four of those a month is a couple of dollars if you notice promptly. And **stopping the API box saves nothing at all** while the free trial lasts, so the cheapest useful posture is that box left running with Valkey and RDS stopped — same $9.26, and the API stays reachable.

Starting Valkey and the database for an afternoon costs under 20 cents. A full load-testing day with Box A upsized to `t4g.large`, both stores up and a `t3.small` driving k6 comes to about **$2.40** — which is why §20 should be run on `large` rather than `medium`: fifty-four more cents buys a result that CPU-credit behaviour has not distorted.

---

20. Load testing

`scripts/load/exam-event.js` is a k6 script covering the five tiers of an exam event, chosen with `-e TIER=`:

Tier	What it simulates
`sitting`	The real thing: log in, start, autosave every 25s, submit
`aftermath`	Everyone opening scorecards, solutions and the leaderboard at once
`login`	The herd arriving in the ten minutes before a test opens
`admin`	An admin watching the live board while it happens
`assets`	Browsers pulling the site and question images

`scripts/bench-scoring.mjs` is separate and measures only the scoring drain, against the database.

Honest status: only `assets` and `login` have ever run, and only against a laptop. The sitting tier — the one that matters — has never been exercised, because it needs student accounts that can reach a startable test.

20.1 Two things to change on the target first

Raise the auth rate limit. It is 120 a minute per IP and a load generator is one IP, so the herd hits that wall long before the box does and your numbers measure the rate limiter.

```
RATE_LIMIT_AUTH_PER_MIN=100000
```

`RATE_LIMIT_SITTING_PER_MIN` needs no change: 60 per student, and an autosave every 25 seconds is two or three. **Put it back afterwards** — a staging box with the limiter off is one somebody can brute-force.

20.2 Making the accounts

This is what has blocked the sitting tier. Staging runs `OTP_SENDER=console`, so the code comes back in the HTTP response — which means a script can create real students over the public API with no SMS provider at all:

```
#!/usr/bin/env bash
set -euo pipefail
API=https://api.staging.examprep.iace.co.in
COUNT=${1:-500}; BASE=9000000000; PIN=1234
echo "mobile,pin" > accounts.csv
for i in $(seq 0 $((COUNT - 1))); do
  MOBILE=$((BASE + i))
  CODE=$(curl -s -X POST "$API/auth/student/otp/request" -H 'content-type: application/json' \
    -H 'x-client: WEB' -d "{\"mobile\":\"$MOBILE\"}" | jq -r '.data.devCode')
  TOKEN=$(curl -s -X POST "$API/auth/student/otp/verify" -H 'content-type: application/json' \
    -H 'x-client: WEB' -d "{\"mobile\":\"$MOBILE\",\"code\":\"$CODE\"}" | jq -r '.data.setupToken')
  curl -s -X POST "$API/auth/student/pin/set" -H 'content-type: application/json' -H 'x-client:
WEB' \
    -d "{\"mobile\":\"$MOBILE\",\"setupToken\":\"$TOKEN\",\"pin\":\"$PIN\"}" > /dev/null
  echo "$MOBILE,$PIN" >> accounts.csv
done
```

Students created this way are `NON_IACE` with no branch, so **they reach nothing yet**. In the admin app: build one test with a real hundred-question paper, put it in a **FREE** series (which every student reaches regardless of branch), set its window open, and **open it once by hand as one of those students**. If one cannot start it, five thousand cannot.

20.3 Running it

Run k6 from a **second small EC2 in** `ap-south-1`, not your laptop. From a laptop you measure your broadband and the path across the internet; from inside the region you measure the box, which is the thing you are sizing. A `t3.small` for an hour is pennies.

```
k6 run -e BASE=https://api.staging.examprep.iace.co.in -e TIER=assets -e ASSET_RPS=20 -e
DURATION=2m scripts/load/exam-event.js
k6 run -e BASE=... -e TIER=login -e LOGIN_PEAK_RPS=40 -e ACCOUNTS=./accounts.csv scripts/load/exam-
event.js
k6 run -e BASE=... -e TIER=sitting -e CANDIDATES=500 -e OPEN_WINDOW_SEC=120 -e SITTING_SEC=600 \
       -e ACCOUNTS=./accounts.csv -e TEST_ID=<the test id> scripts/load/exam-event.js
```

Climb: 100 candidates, then 500, then 1,000. The point of the small run is to find the broken fixture, not the ceiling.

20.4 Reading it

The script carries **thresholds**, so k6 passes or fails on its own terms. They are budgets taken from what the UI tolerates — the paper gives up on a save at 15 seconds and the API refuses at 8:

Measure	Budget
`http_req_failed`	under 1%
`autosave_applied`	over 99%
autosave p95	500 ms
submit p95	2 s
test paper p95	3 s
student login p95	1 s

`throttled_429` **is counted separately on purpose.** A rate-limit rejection is not the API failing, and without that separation a run that hit the limiter reads like a run that fell over. Non-zero means §20.1 was skipped.

Watch `sudo docker stats` and Grafana's queue depth while it runs.

20.5 What the numbers decide

Simulate the event at the intended production size and read the CPU at peak: under 50% and the size is right; 50–80% and go one size up for events; over 80% and go two, and reconsider the container split.

Then **write what you measured into** `docs/04-infrastructure.md` **§16**, which lists every figure that was never measured. The largest is the assumption that a Graviton core is three to four times slower than the laptop these benchmarks came from — every projection in that file rests on it, and one run of `scripts/bench-scoring.mjs` on this box replaces it with a fact.

---

21. The SMS vendor

Until this is done, students cannot sign in on production. It is the longest-lead item here — DLT registration takes **one to two weeks** — so start it even though nothing else depends on it.

India's regulator requires every commercial sender to register on a blockchain-backed registry called DLT. You register three things: your **entity** (the institute, with its GST certificate), your **header** (a six-character sender id such as `IACEDU`), and each **template**, with variables marked. A message that does not exactly match an approved template is rejected by the carrier, not by your code.

1. Pick an aggregator — MSG91, Gupshup, Kaleyra, Textlocal. Ask for **transactional** routing; promotional is blocked on DND numbers and an OTP is not promotional.
2. Register the entity with the institute's GST and PAN.
3. Register the header. **Approval takes days.**
4. Register one template per kind. Every variable is a `{#var#}`:

Kind	Template	Env var
OTP	`{#var#} is your IACE verification code. Valid 5 minutes. Do not share.`	`SMS_TEMPLATE_OTP`
PIN	`Your IACE PIN has been reset. Set a new one at {#var#}`	`SMS_TEMPLATE_PIN`
Result ready	`Your result for {#var#} is ready. IACE`	`SMS_TEMPLATE_RESULT_READY`
Test assigned	`A new test {#var#} is available. IACE`	`SMS_TEMPLATE_TEST_ASSIGNED`
Reminder	`{#var#} opens at {#var#}. IACE`	`SMS_TEMPLATE_TEST_REMINDER`
Announcement	`{#var#}`	`SMS_TEMPLATE_ANNOUNCEMENT`

1. The aggregator gives each approved template an **id**. Those ids go in the env vars, not the text.

```
OTP_SENDER=sms
SMS_PROVIDER_URL=https://api.msg91.com/api/v5/flow/
SMS_PROVIDER_KEY=<auth key>
SMS_SENDER_ID=IACEDU
SMS_TEMPLATE_OTP=<approved id>
SMS_TEMPLATE_PIN=<approved id>
```

The provider is deliberately a **shape, not a name** — endpoint, key, sender id, a template per kind. Every Indian aggregator works this way, so switching vendor is a config change.

OTP and PIN are required; the other four are optional. A kind with no template is treated as switched off and the caller is told, rather than the API recording a message as delivered that was never sent.

The budget is enforced in code. `OTP_GLOBAL_DAILY_BUDGET_PAISE` defaults to ₹3,000 a day and `NOTIFICATION_COST_SMS_PAISE` is 18 paise — a stop on a loop or an attack costing a fortune overnight.

22. Email

An **admin** signs in with an emailed OTP, so email is required the moment `OTP_SENDER` is not `console` — even if SMS is not ready. Gmail through nodemailer's well-known service, so there is nothing to host.

1. The sending account must have **2-Step Verification** on.
2. Google Account → Security → **App passwords** → generate one for Mail. Sixteen characters.

```
MAIL_USER=noreply@iace.co.in
MAIL_PASSWORD=<the 16-character app password>
```

It is not the account password. A normal one is blocked by Google with an unhelpful authentication error.

Beyond a few hundred a day, move to Amazon SES in `ap-south-1` — but SES starts in a sandbox that only sends to verified addresses, and leaving it needs a support request taking a day or two. Gmail is correct for now.

The API refuses to boot with `OTP_SENDER=sms` and no `MAIL_USER`, deliberately: discovering that admins cannot sign in at the first admin login is the worst possible moment.

---

23. When something is wrong

Symptom	Almost always	Check
Caddy gets no certificate	DNS or port 80	`dig +short @8.8.8.8 …`; 80 open to `0.0.0.0/0`; `API_HOST` matches exactly
`migrate` fails on a permission error	`pg_trgm`	§5.2 — create it as master first
`FATAL: too many connections for role`	The role's limit, not the URL's	§5.2 — 85, not 25
API container restarts in a loop	A refused env var	`logs exam` — the refusal names the variable
API refuses: `MEDIA_BASE_URL is required`	It cannot be blank	§7.4
Every question image 404s	The bucket name is in `MEDIA_BASE_URL`	§7.4
`unhealthy` containers	A dependency is unreachable	`curl localhost:3000/health/ready` inside; then the security groups
Valkey `unhealthy`	Password mismatch	`REDISCLI_AUTH` must equal `--requirepass`
API cannot reach Valkey	The URL form	`redis://:PASSWORD@host:6380` — the empty username is required
Site loads, every call fails	`CORS_ORIGINS`	Exact origins, scheme included, no trailing slash. Only in the browser console
Site 404s on a deep link	The CloudFront Function	§11.4
Everything rate-limited	`TRUST_PROXY_HOPS` is 0	They all arrive from Caddy, so they share one bucket
Alloy ships nothing	The two usernames	`PROM_USERNAME` is not `LOKI_USERNAME`
`git` over SSM prints nothing	Dubious ownership, on stderr	`sudo -u ssm-user git -C /opt/examprep …`
`sudo cat > file` denied	The shell opens the file, not sudo	Own the directory, or `sudo tee`
`rand: Extra option`	zsh does not strip `#` comments	Drop the comment from the pasted line
Exit **143**	A clean shutdown	Normal
Exit **137**	SIGKILL	`docker inspect` → `OOMKilled` true is memory; false is the grace period

```
cd /opt/examprep
C="sudo docker compose -f deploy/compose.yml --env-file deploy/.env"
$C ps
$C logs --tail 100 exam
$C logs -f caddy
$C exec exam sh
$C restart exam
sudo docker stats
df -h
```

24. After today

This week	Run the full load test (§20); replace `docs/04` §16's unmeasured figures
This week	Rehearse a point-in-time restore (§17)
This week	Start the DLT registration (§21) — the long pole
Next	Custom domains on the two sites (§12.6)
Next	The four Grafana alerts and the two AWS Budgets (§15.1)
Next	Drop Box A to `t4g.small` once images come from ECR
If the repo goes private	A CodeBuild runner (§13.1), and a `pre-push` hook on `main`
Before production	Decide Box A's size from the load test, and Multi-AZ (§17)
Before production	Plan the `examprep.iace.co.in` cutover from ThinkExam (§3.3)

One habit worth forming. Every number in `docs/04-infrastructure.md` carries the date it was measured and the command that measured it, and §16 lists the ones that are still assumptions. When you measure something, write it there. The largest assumption in the file is that a Graviton core is three to four times slower than the laptop the benchmarks came from — every cost and capacity projection rests on it, and one command on Box A replaces it with a fact.